

Application of Backward Differential Formula and Anderson's method for multigroup diffusion transient equation

Alexey Cherezov*, Alexander Vasiliev, Hakim Ferroukhi

Paul Scherrer Institute, Switzerland

ARTICLE INFO

Keywords:

Finite element method
B-splines
Backward Differential Formula
Anderson method
Diffusion equation
Transient simulation

ABSTRACT

This paper focuses on the application of the Backward Differential Formula (BDF) and Anderson's method for neutron diffusion transient simulations. The B-spline finite element method is used for spatial discretization of continuous equations into the system of ODEs. BDF with adaptive step size and approximation order is used for integration of the ODEs over time. In-step multigroup equation is solved using the block Gauss–Seidel method, where the inner equation is solved using the ILU(0)-preconditioned Conjugate Gradient method. Anderson's method is employed first to solve the in-step non-linear equation that couples the neutron and temperature models, and second, to accelerate the convergence of the block Gauss–Seidel algorithm. The performance of the time integration scheme and the in-step equation solver are verified on the point kinetics equation and two-dimensional benchmarks.

1. Introduction

The finite element method (FEM) is a numerical technique for solving partial differential equations through piece-wise polynomial or spline approximation. The method is advantageous for reactor physics as it can accommodate various reactor core models, including multigroup diffusion, spherical harmonics, and discrete ordinates, and can be applied to a range of reactor geometries and geometry changes. The method enables automatic refinement of the solution by increasing the number of mesh points and/or polynomial order to guarantee the required solution accuracy (Wang and Ragusa, 2009). In addition, it yields solutions that are compatible with other fields of physics, such as fluid dynamics and fuel thermal-mechanics, where the finite element approximation is extensively used, thus reducing the efforts required for codes coupling. Finally, the FEM discretization of physical equations results in large sparse matrices, which can efficiently be treated using powerful iterative schemes (Saad, 2003) and high performance numerical libraries. These benefits make FEM practically useful for many reactor physics applications, for example, small and micro-reactors (Bousquet et al., 2020), molten-salt reactors (Jaradat et al., 2024), mechanical vibrations of fuel assemblies (Vidal-Ferrándiz et al., 2020a), and reactors with hexagonal geometry (Hébert, 2008; González-Pintor et al., 2009).

Numerous finite element (and finite volume) algorithms have been developed for the solution of time-dependent neutron transport and diffusion equations, including MINARET (Baudron et al., 2014) and MINOS (Baudron and Lautard, 2007) in APOLLO-3 (Greiner et al.,

2023), Rattlesnake (Wang et al., 2021), Griffin (Lee et al., 2021), FEMFUSION (Vidal-Ferrándiz et al., 2020a,b), FENNECS (Io Muzio and Seubert, 2024; Bousquet et al., 2020), GenFOAM (Fiorina et al., 2016, 2022), CAST3M (Kepisty et al., 2017), SPNDYN (Babcsányi et al., 2020), ntkFoam (Ma et al., 2021), TASNAM (Ge et al., 2015), FLUENT (Zhang et al., 2020) and many other. Generally, these codes employ the implicit time-step integration schemes, which guarantee the stability during the numerical integration process (Hairer et al., 1987; Hairer and Wanner, 1996). However, implicit methods require inversion of a large system of linear equations at each time step. The neutron kinetics system is poorly conditioned due to the large variations in neutron free mean paths and lifetimes, resulting in slow convergence of the iterative linear system solvers. Therefore, developing efficient solution algorithms for the in-step system of equations remains one of the primary challenges in creating an efficient neutronics transient solver.

Reactor codes typically combine neutronics, thermal-hydraulics and fuel performance models interconnected through various physics feedbacks for steady-state and transient simulations (Cherezov et al., 2020). The numerical algorithm for time-dependent models is nested and involves global, constituent, and inner iterations (Senecal and Ji, 2018). The global iteration represents the highest level, advancing time steps to ensure convergence of the entire transient process, following the implicit time integration scheme. Constituent iteration, within a time step, addresses the reactor models at specific time moments and continues until the feedback parameters of the models converge; convergence is usually enhanced by Picard's method with a damping coefficient or

* Corresponding author.

E-mail address: alexey.cherezov@psi.ch (A. Cherezov).

Jacobian-Free Newton–Krylov (JFNK) method (Hamilton et al., 2016). Inner iteration, within each constituent model, focuses on solving equations specific to a particular physics domain.

The constituent iteration refers to a non-linear equation that can be solved iteratively by Picard method, starting from an initial guess being close enough to the true solution. However, the method has a linear convergence rate, and in general the convergence is not guaranteed. Therefore, the Newton–Raphson method can be employed, which is known to exhibit quadratic convergence. However, this method relies on the usage of the Jacobian matrix, which computation is often impractical for large scale applications. To address this issue, quasi-Newton methods have been developed, analogous to the secant method in one-dimensional space. The Jacobian matrix in these methods is approximated using relatively crude techniques, such as finite difference quotients or first-order Taylor expansions. Though the convergence of quasi-Newton methods is generally slower than that of exact Newton methods, their overall performance can be superior since the exact evaluation of the Jacobian is not required. Furthermore, for certain methods like Anderson’s method (Eyert, 1996) and Jacobian-Free Newton–Krylov (JFNK) method, the Jacobian matrix does not need to be explicitly constructed and inverted, that contributes to an overall improvement in computational efficiency.

The inner iteration of the multi-group diffusion neutron kinetics model involves multiple levels. For the state-of-the-art approach, the multi-group system of equations is solved group-by-group, resulting in the block Gauss–Seidel (GS) method. However, the GS method requires strict diagonal dominance of the multi-group system to ensure iteration convergence (Saad, 2003). In the implicit step scheme, diagonal dominance is satisfied by the term τ/h , where h denotes the step size and τ represents the prompt neutron lifetime varying from 10^{-7} s to 10^{-2} s. Consequently, if a step size is as large as $h \sim 1$ s, dominance is not guaranteed and convergence may be slow and even not reached. To address this issue, various acceleration techniques, such as Chebyshev (Zimin and Ninokata, 1996) and Aitken (Fiorina et al., 2022) methods, are employed. The alternatives to the GS approach are preconditioned Conjugate–Gradient (CG) like methods applied to the non-symmetric multi-group matrix (Yang et al., 1993; Gupta et al., 2005), including Generalized Minimum Residual (GMRES) method (Al Daas et al., 2018) and Stabilized Bi–Conjugate Gradient (BiCGStab) method (Vidal-Ferrándiz et al., 2022), a non-linear acceleration technique based on two energy grids (Anistratov et al., 2017), convolutional neural networks (Phillips et al., 2023) and advanced sparse Gaussian elimination methods (Grigori et al., 2008; Scott and Tuma, 2023).

In the block GS method, the inner within-group iteration refers to inversion of a Symmetrical Positive Definite (SPD) (Saad, 2003) matrix, which is typically computed by the CG method with the preconditioning matrix based, for example, on incomplete LU (ILU) factorization, successive over-relaxation (SOR), incomplete Cholesky (IC) factorization, or algebraic multi-grid (AMG) method. Some of the codes implement the geometric domain decomposition method for memory reduction (Vidal-Ferrándiz et al., 2017) and parallel execution of large-scale reactor core problems on multi-processors.

In this paper, we introduce a new algorithm, FEMCORE, developed for solving the time-dependent multi-group neutron diffusion equation. The algorithm utilizes FEM integrands generated for a specific reactor domain by a third-party FEM package, such as NUTILS (van Zwieten et al., 2022) or others, as input. It then assembles the FEM matrices for specific cross-sections and performs integration using the same solution scheme employed by other codes. One key feature of the algorithm is its implementation of Anderson’s method for solving the non-linear system of in-step multi-physics equations and for enhancing convergence of the in-step neutron diffusion equation, which is iterated by the GS method. Anderson’s method has shown effectiveness for multi-physics equations and reactor core eigenvalue problems, prompting us to assess its efficiency also in solving the in-step neutronic equation. A comparative study on the Picard, Anderson and JFNK methods for

steady-state problems is presented by Hamilton et al. (2016). Application of Anderson’s method for neutron transport eigenvalue problem was considered by Calloo et al. (2023) and Shen and Kochunas (2024).

FEMCORE utilizes the Backward Differential Formula (BDF) (Hairer and Wanner, 1996; Gear, 1971) with automatic step size and integration order selection for time integration. There are two widely-used implicit multi-step methods, BDF and Adams’, being stable up to the 6th and 12th orders, respectively. The stability region of the Adams’ method contains large but finite part of the left half of the complex plane, allowing the use of very large variation in step sizes compared to explicit methods. In contrast, the stability region of BDF includes the entire of the negative real axis, allowing steps of any size. Consequently, BDF is considered the most powerful method for handling stiff problems. It found numerous applications and proven efficient in reactor physics (Ginestar et al., 1998; Shim et al., 2011; Tiberge et al., 2020). However, many researchers use only low order BDF methods (Bru et al., 2002; Cai et al., 2019; Tiberge et al., 2020) and/or without automatic control. Thus, analyzing the BDF method for reactor applications remains important. A potential alternative to implicit methods is the use of explicit stabilized methods (Abdulle, 2015). These methods extend the stability region along the negative real axis, allowing large time steps for integrating diffusion PDEs without solving a linear or nonlinear problem at every step.

BDF fits solution vectors computed at previous time steps. The derivation of this method is based on backward differences, which approximate the higher-order derivatives using function values at the current and previous points, combined with binomial coefficients (Hairer and Wanner, 1996). The extensive study of the BDF method one can find in the monograph of Gear (1971) and in the book of E. Hairer (Hairer et al., 1987). The BDF method utilized in this work is based on the efficient formulation proposed by A. Nordsieck in 1962 (Nordsieck, 1962) and implemented in the LSODE package (Radhakrishnan and Hindmarsh, 1993), with adaptations for solving the in-step equation using Anderson’s method. This approach enables high-order approximation of the time-dependent solution, including both the neutron flux and delayed neutrons precursors. Therefore, FEMCORE achieves direct integration of the neutron kinetic equations, avoiding semi-explicit integration of the precursor equations, thus increasing the efficiency of implementation.

The rest of the paper is organized as follows. Section 2 introduces the multi-group diffusion equation and discretization of this equation by FEM, the BDF algorithm with automatic step and order selection, the iterative methods used for solution of the in-step multi-group equation, and the Anderson’s method. Section 3 presents the numerical results for the point-kinetic non-linear test SKINATH, and two-dimensional transient benchmarks TWIGL and LRA, and performs the analysis of FEMCORE convergence and performance on the BDF and FEM orders. Section 3 contains the study conclusions.

2. Methodology

In this section, we describe the multigroup diffusion transient equation and the methods used for its numerical solution, including the B-spline FEM, the BDF method, and various linear system solvers. The in-depth information on the neutron diffusion equation can be found in nuclear engineering handbooks (Cacuci, 2010; Stacey, 2007; Lewis, 2008; Lamarsh, 1966). The theory and practical aspects of FEM one can find in the textbooks (Brenner and Scott, 1994; Babuska and Strouboulis, 2001; Zienkiewicz et al., 2013; Ern and Guermond, 2004; Brenner and Carstensen, 2004). A general derivation of FEM equations for multigroup diffusion model one can find in Cacuci (2010), Wang and Ragusa (2009), González-Pintor et al. (2009), and a very detailed explanation of the B-Spline FEM is given by Ferguson et al. (2022). To represent the equations in a compact form, we use the Einstein-like summation notation adapted to the multidimensional arrays. The notation is intuitive and commonly used in the computation science community. However, to mitigate any confusion, the rules for this notation are listed in Appendix A.

2.1. Model description

2.1.1. Neutron diffusion equation

The time-dependent neutron diffusion equation reads

$$\begin{cases} \frac{1}{v_g} \frac{\partial \Phi_g}{\partial t} = -\frac{\partial J_g^n}{\partial x_n} - \Sigma_g \Phi_g + S_{gg'} \Phi_{g'} + (1 - \beta) \chi_g F_{g'} \Phi_{g'} + \chi_g^m \lambda^m C_m, \\ J_g^n = -D_g \frac{\partial \Phi_g}{\partial x^n}, \end{cases} \quad (1)$$

where the summation over indices g' , n and m in the first equation is assumed. Here $t > 0$ is time, x_n are spatial coordinates, $\Phi_g = \Phi_g(t, x_n)$ is the multi-group scalar neutron flux, $J_g^n = J_g^n(t, x_n)$ is the vector of neutron current, v_g is the neutron velocity that depends only on the energy group, $D_g = D_g(t, x_n)$ is the diffusion coefficient, Σ_g is the array of removal cross-sections $\Sigma_r^g(t, x_n)$ from group g by scattering and absorption, $S_{gg'}$ is the array of scattering cross-sections $\Sigma_s^{g' \rightarrow g}(t, x_n)$ from group g' into group g , F_g is the array of fission production cross-sections $\nu \Sigma_f^g(t, x_n)$, χ_g and χ_g^m are the fission spectra for prompt and delayed neutrons that depend only on energy group g and delayed neutron family m , β is the effective fraction of delayed neutrons, V is the reactor volume.

The time-dependent equations for neutron precursor densities reads,

$$\frac{\partial C_m}{\partial t} = \beta_m F_{g'} \Phi_{g'} - \lambda_m C_m, \quad (2)$$

where λ_m is the decay constant of precursor of family m , β_m is the fraction delayed neutrons, and $C_m = C_m(t, x_n)$ represents the neutron precursor density. Indices g and m stand for the energy group and precursor family.

2.1.2. Boundary conditions

The equation is solved with the albedo boundary conditions on the surface ∂V of reactor domain V where neutrons undergo reflection or absorption when they are incident on a scattering or absorbing medium. Albedo γ is defined as the incoming current of the reflected neutrons divided by the outgoing current of the incident neutrons. In the multigroup model, it is represented as a matrix that couples the neutron currents across different energy groups. In this work, we assume that the albedo coefficient applies independently to each energy group, indicating no coupling. Denote $\gamma_g = J_g^- / J_g^+$, where the partial incoming and outgoing currents J_g^- and J_g^+ projected onto normal vector N_n are calculated as follows

$$J_g^\pm = \frac{1}{4} \Phi_g \mp \frac{1}{2} D_g N_n \frac{\partial \Phi_g}{\partial x^n}. \quad (3)$$

Here N_n is the unit vector pointing out of the boundary ∂V . The boundary conditions are written as follows

$$\frac{1}{2} \alpha_g \Phi_g + N_n J_g^n = 0 \text{ on } \partial V, \quad (4)$$

where $\alpha_g = (1 - \gamma_g) / (1 + \gamma_g)$. The reflective boundary conditions correspond to $\gamma_g = 1$ and $\alpha_g = 0$, while the vacuum boundary conditions correspond to $\gamma_g = 0$ and $\alpha_g = 1$. Zero-flux boundary conditions are equivalent to $\alpha_g \rightarrow \infty$.

2.2. Finite Element Method

The Finite Element Method (FEM) involves three steps: (a) weak formulation of the differential equation; (b) discretization of the problem domain into finite elements; and (c) representation of the differential equation in matrix form. The following subsections apply these steps to the diffusion neutron kinetics equations.

2.2.1. Weak formulation

Consider a Hilbert (Sobolev) space H^1 of continuous, once-differentiable functions that are square-integrable over the re-

actor domain V , together with their first-order derivatives. We seek the functions $\Phi_g(t)$ and $C_m(t)$ in H^1 for each time t , energy group g and precursor family m . To derive the variational form of the neutron kinetics equations, we multiply each of the equations by an arbitrary function $\Psi \in H^1$, and then integrate them over V . The integrals exist due to the integrability of the functions from H^1 . However, this functional space does not state the continuity of the net neutron current J_g^n , which derivative is therefore undefined. To resolve this inconsistency, the diffusion operator in Eq. (1) is integrated by parts as follows,

$$\int_V \Psi \frac{\partial J_g^n}{\partial x_n} dV = - \int_V J_g^n \frac{\partial \Psi}{\partial x_n} dV + \int_{\partial V} \Psi J_g^n N_n dS, \quad (5)$$

where dS is the elementary surface of the reactor boundary ∂V that corresponds to the direction N_n . This transformation reduces the requirements on the smoothness of the neutron current, and introduces an additional term related to the neutron leakage. Moreover, this equation implicates the continuity of the net current J_g^n at an arbitrary elementary surface Γ inside V , including the interfaces of finite elements. That is for an arbitrary weight function $\Psi \in H^1$, the following equation is valid

$$\int_{\Gamma^-} \Psi J_g^n N_n^- dS = \int_{\Gamma^+} \Psi J_g^n N_n^+ dS, \quad (6)$$

where the sign $(\pm)$ indicates two sides of the elementary surface Γ . Note, this equation is automatically satisfied as soon as the weight function Ψ is continuous on Γ , and no additional terms are required in the weak equation. However, for the discontinuous FEM, the additional terms have to be introduced via trace operators (Ragusa, 2015; Hackemack, 2021) used to couple the solution at interior edges of the finite elements.

The expression under the surface integral in Eq. (5) is substituted by the boundary conditions given by Eq. (4),

$$\int_{\partial V} \Psi J_g^n N_n dS = -\frac{\alpha_g}{2} \int_{\partial V} \Psi \Phi_g dS. \quad (7)$$

Using the above transformations, we finally obtain the weak form of the neutron kinetics equations for each energy group g , which reads

$$\begin{aligned} \frac{1}{v_g} \frac{\partial}{\partial t} \int_V \Psi \Phi_g dV &= \int_V D_g \frac{\partial \Phi_g}{\partial x^n} \frac{\partial \Psi}{\partial x_n} dV \\ &\quad - \frac{\alpha_g}{2} \int_{\partial V} \Psi \Phi_g dS - \int_V \Sigma_g \Phi_g \Psi dV + \\ &\quad + \int_V S_{gg'} \Phi_{g'} \Psi dV + (1 - \beta) \chi_g \int_V F_{g'} \Phi_{g'} \Psi dV \\ &\quad + \chi_g^m \lambda^m \int_V C_m \Psi dV, \end{aligned} \quad (8)$$

The variational form of the precursor equation of family m is derived straightforward,

$$\frac{\partial}{\partial t} \int_V C_m \Psi dV = \beta_m \int_V F_{g'} \Phi_{g'} \Psi dV - \lambda_m \int_V C_m \Psi dV. \quad (9)$$

These equations must hold for any $\Psi \in H^1$. Under the appropriate conditions of H^1 , a unique solution to the variational equations exists (Brenner and Scott, 1994). The variational formula can be converted to a matrix-vector form using a finite subset of functions from space H^1 , as will be discussed in the next section.

2.2.2. Basis functions

FEM approximates the solution by a finite set of basis functions defined on the support regions, combined of rectangles and triangles in 2D geometry. The support regions are overlapped (continuous methods) or coupled via average/jump operators (discontinuous methods) leading to the sparse linear system of equations. The number of the basis functions define the number of model unknowns or the degrees of freedom (DOF) of the solution.

In this study, we used the FEM with B-splines (Holling, 2003). The C^0 -smooth requirement is imposed at the elementary physical regions

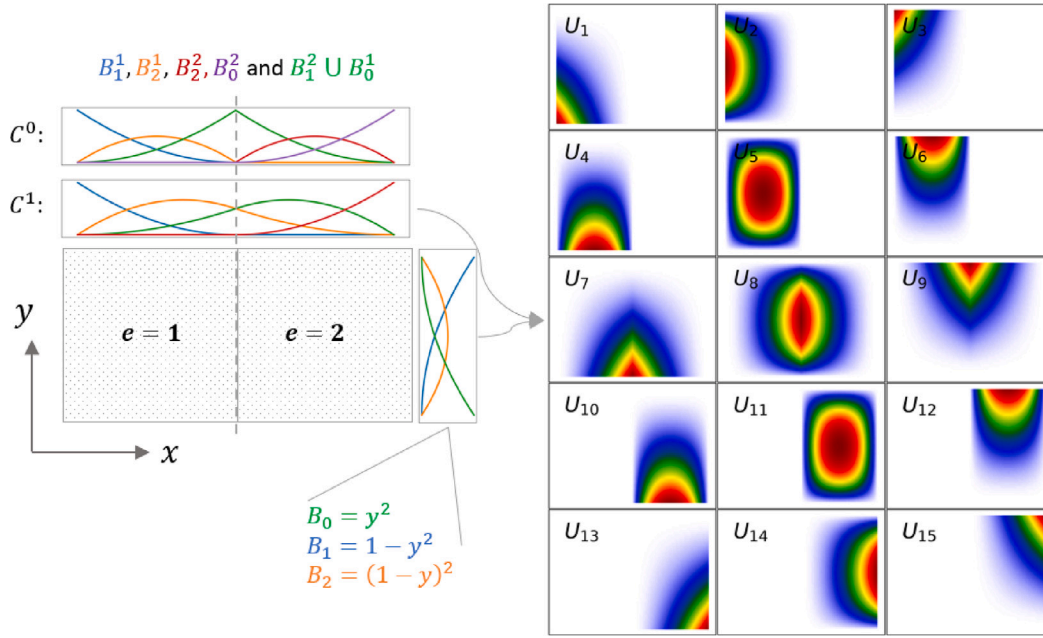


Fig. 1. B-spline basis for two-region problem.

with the constant compositions, to ensure the continuity of the neutron flux. The basis functions $u_i = u_i(x_n)$, calculated by the tensor product of the B-splines (Piegl and Tiller, 1997), form the finite-dimensional subspace in $H^1(V)$ where the neutron diffusion equation is discretized. The introduction into the B-Spline FEM with all necessary technical details is given in Holling (2003), Aggarwal (2006).

Lets illustrate the construction of second-order B-spline basis and shape functions in a 2D geometry for an elementary problem with two subdomains, as shown in Fig. 1. Each subdomain contains 3 splines along each direction, denoted by B_k^e , where $e = 1, 2$ is the subdomain index and $k = 0, 1, 2$ is the 1D-spline index. The splines take non-zero values only within their respective subdomains e and continuously extended to zero on an opposite subdomain. However, splines B_0^1 and B_1^2 from subdomains 1 and 2 take non-zero values at the interface; hence, they are concatenated at the interface point to ensure continuity of the final solution. Thus, there are 5 basis functions along the x_1 axis and 3 basis functions along the x_2 axis. Increasing the smoothness of basis functions from C^0 to C^1 reduces the number of basis functions. The tensor product of 1D basis functions produces 15 basis functions u_i for the 2D problem, as illustrated on the right side of the graphic.

2.2.3. Matrix assembling

The expansion of the group neutron flux and the concentration of the delayed neutron precursors over the finite set of B-spline shape functions u_i reads

$$\Phi_g(t, x_n) = \Phi_{ig}(t) u^i(x_n), \quad (10)$$

$$C_m(t, x_n) = C_{im}(t) u^i(x_n), \quad (11)$$

where Φ_{ig} and C_{im} are the unknown coefficients to be determined by solving a linear system of equations. To assemble this system of equations, we come back to the variational form of the neutron kinetics equations given by Eqs. (8) and (9) and integrate them for the arbitrary function Ψ running over each shape function v_j . In this work, we assume that the expansion and weight functions u_i and v_j are selected from the same finite set and do not vary with the energy group. Doing this, we assume the model parameters are constant across each rectangular finite element e , and each finite element associated with a specific diffusion coefficient D_g^e , removal term Σ_g^e , scattering term

$S_{gg'}^e$, and fission production term F_g^e . Then, the integral over removal cross-section is written as follows

$$\int_V \Sigma_g u^i v^j dV = \Sigma_g^e \int_e u^i v^j dV = \Sigma_g^e \epsilon_e^{ij}. \quad (12)$$

In this equation, the removal cross-sections Σ_g^e of group g are summed with term ϵ_{ij} over the finite elements e . The integrals for $S_{gg'}$ and F_g terms undergo the similar transformations, resulting to the same integrals ϵ_e^{ij} referring to neutron-nuclide collisions. The diffusion and surface terms, however, are transformed in a different way,

$$D_g^e \int_V \frac{\partial u^i}{\partial x_n} \frac{\partial v^j}{\partial x_n} dV + \frac{\alpha_g}{2} \int_{\partial V_e} v^j u^i dS = D_g^e \delta_e^{ij} + \frac{\alpha_g}{2} \sigma^{ij}, \quad (13)$$

where the volumetric integrals δ_e^{ij} refer to the neutron diffusion, and the surface integrals σ^{ij} refer to the neutron leakage.

Fig. 2 illustrates the structure of the collision, leakage, and diffusion matrices ϵ^{ij} , σ^{ij} and δ^{ij} for the two-subdomain problem discretized by the second-order B-spline FEM. The matrices ϵ^{ij} and δ^{ij} are obtained by summing the corresponding terms over the finite element index e . The column and row indices of the matrices correspond to specific 2D shape function shown in Fig. 1. The collision and leakage matrices have positive entries (colored orange), while the diffusion matrix contains many negative elements on the side diagonals (colored blue) associated with the gradients of the basis functions. The leakage matrix has non-zero entries in all columns and rows except for the shape functions u_5 , u_8 , and u_{11} taking zero at the problem boundary. The subdomains interconnect in the collision and diffusion matrices through the shape functions u_7 , u_8 and u_9 associated with the subdomains interface.

From Eqs. (12) and (13) it follows that the parameters of the multi-group equation are separated into geometric and physical components. The geometric parameters include volumetric and surface integrals over polynomial basis functions, represented by terms ϵ_e^{ij} , δ_e^{ij} , and σ^{ij} calculated only one time for a given mesh using automatic tools. The physical parameters include the diffusion and cross-section coefficients D_g^e , Σ_g^e , $S_{gg'}^e$ and F_g^e , which should be updated for each constituent iteration to account for the temperature feedback, change of the nuclide number densities and/or movement of control rods. The geometric and physical terms are assembled into the finite element matrices,

$$D_g^{ij} = D_g^e \delta_e^{ij}, \quad \Sigma_g^{ij} = \Sigma_g^e \epsilon_e^{ij}, \quad S_{gg'}^{ij} = S_{gg'}^e \epsilon_e^{ij}, \quad F_g^{ij} = F_g^e \epsilon_e^{ij}, \quad (14)$$

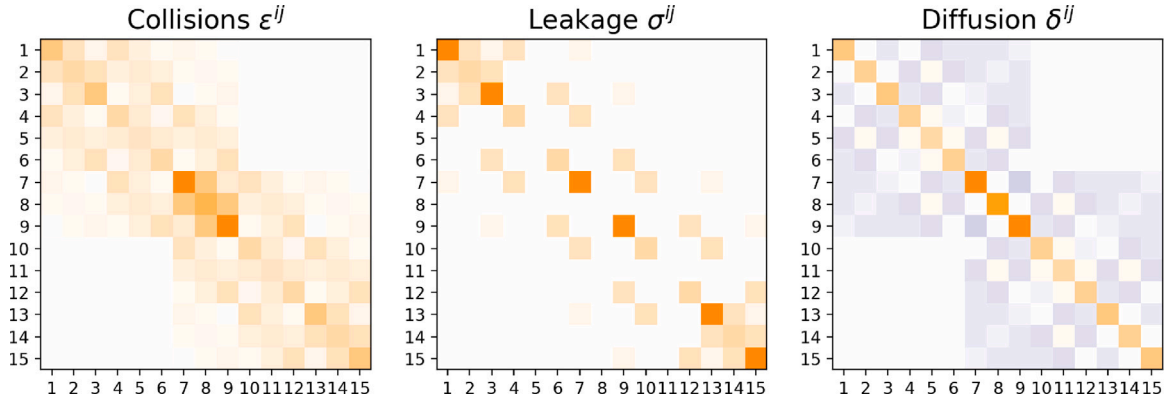


Fig. 2. FEM matrices example.

for each energy group g and g' . The dimension of these matrices is equal to the total number of degrees of freedom. Moreover, the terms δ_e^{ij} and ϵ_e^{ij} are the sparse SPD matrices for each element e since they correspond to the symmetric diffusion and collision operators, and the expansion and weight functions are chosen from the same finite-dimensional space.

2.2.4. ODEs

Using the discretization formula from the previous section, the neutron kinetics equations take the form

$$\frac{\epsilon^{ij}}{v_g} \frac{d\Phi_{ig}}{dt} = -L_g^{ij} \Phi_{ig} + S_{gg'}^{ij} \Phi_{ig'} + (1 - \beta) \chi_g F_g^{ij} \Phi_{ig'} + \chi_g^m \lambda^m R_m^j, \quad (15)$$

$$\frac{dR_m^j}{dt} = \beta_m F_g^{ij} \Phi_{ig'} - \lambda_m R_m^j, \quad (16)$$

where L_g^{ij} denotes the matrix of neutron loss by leakage and absorption,

$$L_g^{ij} = D_g^{ij} + \frac{\alpha_g}{2} \sigma^{ij} + \Sigma_g^{ij}. \quad (17)$$

A new vector of unknowns is introduced by $R_m^j = \epsilon^{ij} C_{im}$ to replace the vector of precursor density C_{im} . The motivation is that precursors are usually not the quantities of interest in reactor physics, and they are calculated to provide the right source of delay neutrons in the multigroup equation. Therefore, in order to reduce computational costs associated with the inversion of the mass matrix ϵ^{ij} , the vector of precursor densities is replaced by vector R_m^j , in which the sum of elements over index i is equal to the total amount of precursors of family m . The initial value of this vector can be found from the steady-state conditions, by setting the left-side derivative of Eq. (16) to zero.

The system of equations given by Eqs. (15) and (16) is stiff, as it describes components of the neutron flux and precursors that change rapidly and slowly over space and time. Consequently, explicit time integration schemes are unsuitable due to their restricted stability regions, which necessitate small time steps and result in very slow computational performance. Therefore, we employ an arbitrary order implicit method for integrating this system, as described in the next section.

2.3. BDF method

It has been noted that the multi-step methods are superior in accuracy of the one-step methods “since they permit a better approximating curve to be fitted over the elementary interval” (Nordsieck, 1962). An implicit multi-step integration method proceeds in two steps: firstly, it predicts the solution based on polynomial extrapolation of the previous solution vectors, and secondly, it corrects the predicted vector to ensure that the derivative of the approximating curve corresponds to the right-hand side function. Here, we formulate the BDF multi-step method using the Nordsieck’s approach, which simplifies the prediction step and facilitates adjustments of the algorithm to the new method order and step size.

2.3.1. Recurrence formula

The system of ordinary differential equations reads

$$\frac{dy_i}{dt} = f_i(t, y_1, \dots, y_n), \quad (18)$$

where index i varies from 1 to n , $t \geq 0$ is the time parameter, $y_i = y_i(t)$ is the vector of unknowns, $y_i(0) = y_{i0}$ is the initial vector, and f_i is in general a non-linear vector function. Here, functions $y_i(t)$ are assumed to be q times differentiable over t .

Consider the elementary time interval $[0, h]$ associated with the step size h . The Taylor’s expansion of $y_i(t)$ on this interval at $t = 0$ is given by

$$y_i(t) = p_i(s) + O(h^{q+1}), \quad (19)$$

where p_i is the q -order polynomial and $s = t/h$ is the dimensionless parameter that varies from 0 to 1 on the elementary interval $[0, h]$. Following the Nordsieck’s formulation, denote z_i^k are the coefficients of $p_i(s)$, and $\tau_k(s) = s^k$ are the monomial functions. Then, polynomial $p_i(s)$ approximating function $y_i(t)$ on the elementary interval reads

$$p_i(s) = z_i^k \tau_k(s), \quad (20)$$

where summation over index k is assumed. The polynomial coefficients, known in the BDF terminology as the Nordsieck’s vector, are computed as follows

$$z_i^k = \frac{h^k}{k!} \frac{d^k}{dt^k} y_i(t), \text{ at } t = 0, \quad (21)$$

for each derivative index $k = 0, \dots, q$ and component index $i = 1, \dots, n$. These coefficients fully determine the behavior of the unknown function $y_i(t)$ on the elementary interval h within accuracy given by Eq. (19). To numerically integrate Eq. (18), we need to establish a recurrent formula to update these coefficients for transitioning from one step interval to the next.

At the very first step, the first two components of the Nordsieck’s vector are given by the initial conditions, that is $z_i^0 = y_{i0}$ and $z_i^1 = h f_i(0, y_{i0})$. Other components are usually not determined, but can be set to zero if the initial steady-state is considered, that is $z_i^k = 0$ for $k = 2, \dots, q$. However, all the components of the Nordsieck’s vector must be corrected to capture the actual behavior of function $f_i(t)$ and to maintain numerical stability. To achieve this, we introduce an arbitrary q -degree polynomial $r(s)$ on the interval $[0, h]$, which is used to correct the polynomials $p_i(s)$ in the following manner,

$$u_i(s) = p_i(s) + \beta_i r(s) = (z_i^k + \beta_i l^k) \tau_k(s), \quad (22)$$

where summation over index $k = 0, \dots, q$ is assumed. In this equation, l^k are the fixed coefficients referred to the correction polynomial $r(s) = l^k \tau_k(s)$, β_i are the correction coefficients that depend on step size h , and $u_i(s)$ are the updated approximating polynomials. Note, that a correction coefficient β_i must be as small as at least $O(h^q)$ to maintain order q of the corrected approximation u_i .

A correction coefficient β_i can be expressed from the equation above as follows,

$$\beta_i = \frac{1}{r} (u_i - z_i^k \tau_k), \quad (23)$$

where $r = r(1)$, $u_i = u_i(1)$ and $\tau_k = \tau_k(1)$ are the functions at the end of the elementary interval, for brevity. The corrected polynomial $u_i(s)$ must satisfy Eq. (18) at the end of the interval $[0, h]$, that is

$$\dot{u}_i = hf(h, u_i). \quad (24)$$

where $\dot{u}_i = du_i(s)/ds$ at $s = 1$. To compute unknown vector u_i , take the derivative of $u_i(s)$ given by Eq. (22) over s , substitute it into Eq. (24) assuming $s = 1$, and use Eq. (23) to eliminate β_i . Then, the final equation for u_i reads

$$u_i - \alpha hf_i(h, u_i) = \tau_k z_i^k - \alpha \dot{\tau}_k z_i^k, \quad (25)$$

where summation over index $k = 0, \dots, q$ is assumed on the right hand side. Here, the terms τ_k and $\dot{\tau}_k$ are equivalent to vectors $(1, 1, 1, \dots, 1)^T$ and $(0, 1, 2, \dots, q)^T$, respectively, and $\alpha = r/\dot{r}$ is a constant that depends on the coefficients l_i of the correction polynomial $r(s)$, since $r = r(1) = l^k \tau_k$ and $\dot{r} = l^k \dot{\tau}_k$. Once the solution vector at the end of the elementary interval u_i is found, the correction coefficients β_i are found immediately from Eq. (23), and the correction terms to the Nordsieck's vector z_i^k are computed according to Eq. (22). We expect that the corrected polynomials are continuously extend each other over the time intervals, that is $u_i(s) = \tilde{u}_i(s)$. Therefore, the correction polynomial $r(s)$ must be always equal to zero at the beginning of an interval, that is $r(0) = 0$ and, hence, $l_0 = 0$.

For the next iteration step, polynomial $u_i(s)$ is extrapolated on the elementary interval $[h, 2h]$ followed by $[0, h]$. Let us write the extrapolation procedure in a recurrent form. Note, that

$$\tau_k(s+1) = P_k^j \tau_j(s), \quad (26)$$

where summation over index $j = 0, \dots, q$ is assumed, and P_k^j denotes the lower triangular Pascal matrix, which contains the binomial coefficients as its elements emerged after expansion of terms $(s+1)^k$ in the vector $\tau_k(s+1)$. As an example, the above equation for $q = 6$ reads

$$\begin{bmatrix} (1+s)^0 \\ (1+s)^1 \\ (1+s)^2 \\ (1+s)^3 \\ (1+s)^4 \\ (1+s)^5 \\ (1+s)^6 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 1 & 0 & 0 & 0 & 0 \\ 1 & 3 & 3 & 1 & 0 & 0 & 0 \\ 1 & 4 & 6 & 4 & 1 & 0 & 0 \\ 1 & 5 & 10 & 10 & 5 & 1 & 0 \\ 1 & 6 & 15 & 20 & 15 & 6 & 1 \end{bmatrix} \begin{bmatrix} s^0 \\ s^1 \\ s^2 \\ s^3 \\ s^4 \\ s^5 \\ s^6 \end{bmatrix}. \quad (27)$$

Using this recurrent formula, the extrapolation of polynomial $u_i(s)$ given by Eq. (22) to the next step can be written as follows,

$$u_i(s+1) = (z_i^k + \beta_i l^k) P_k^j \tau_j(s) = \tilde{z}_i^j \tau_j(s) = \tilde{p}_i(s), \quad (28)$$

that is the extrapolated value of the polynomial after correction u_i at the previous time step is equal to the polynomial before correction $p_i'(s)$ at the following time step. Here, $\tilde{z}_{ik}$ and $\tilde{p}_i$ denote the Nordsieck's vector and polynomial at the new time interval, respectively. Thus, the relation between solutions calculated at the previous and the next time steps is given by

$$\tilde{z}_i^j = P_k^j (z_i^k + \beta_i l^k). \quad (29)$$

Fig. 3 illustrates the BDF curve built for the initial value problem, $\dot{y} = f(t, y)$ and $y(0) = y_0$, over two consecutive intervals $[0, h]$ and $[h, 2h]$. The approximation curve begins with the initial value y_0 and the first order derivative $f(0, y_0)$ computed at time $t = 0$. The predictive curve $p_0(s)$ associated with the Nordsieck vector z_0^k is extrapolated to the end of the first interval $t = h$. The components z_0^k are corrected by $\beta_0 l^k$ such that the derivative of the resulting polynomial $q_0(s)$ coincide with the value $hf(h, q_0(h))$. To find such β_0 , the non-linear equation given by Eq. (25) is solved. Then, using the transformation based on

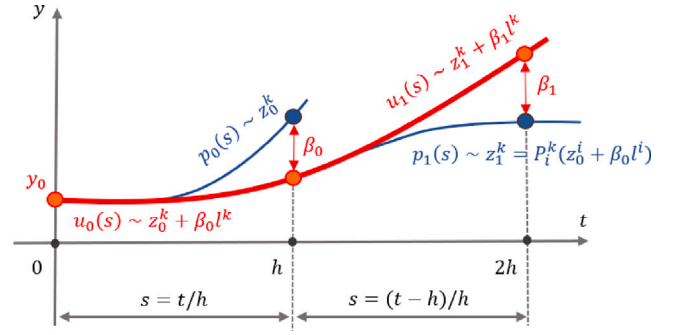


Fig. 3. Time stepping of the BDF method.

the Pascal matrix as shown by Eq. (26), the corrected polynomial $q_0(s)$ is extrapolated to the new interval $[h, 2h]$, where it is denoted by $p_1(s)$ and associated with vector z_1^k . Then, $p_1(s)$ is again corrected by β_1 to obtain $u_1(s)$, and so on, until the end of the simulation interval.

2.3.2. Stability requirements

The method's stability depends on the choice of coefficients l_k . As it is usually done in the stability analysis of the time integration methods (Hairer et al., 1987), let us consider BDF scheme for the Dalquist problem with $f(t, y) = \lambda y$ and λ is an arbitrary complex value. Then, substituting u_i from Eq. (25) to Eq. (23), the correction coefficient β_i reads

$$\beta_i = -\frac{1}{\dot{r} - \omega r} (\dot{\tau}_k - \omega \tau_k) z_i^k, \quad (30)$$

where $\omega = \lambda h$ and the summation over index $k = 0, \dots, q$ is assumed. Substituting this equation into Eq. (29), the recurrent formula for the Nordsieck vector is written as follows,

$$\tilde{z}_i^k = P_m^k \left(\delta_j^m - \frac{\dot{\tau}_j - \omega \tau_j}{\dot{r} - \omega r} l^m \right) z_i^j \equiv M_j^k(\omega) z_i^j, \quad (31)$$

where δ_j^m is Kronecker's symbol and summation over indices m and j is assumed. The propagation matrix $M(\omega)$ depends on parameter $\omega = \lambda h$ and coefficients l^k . For stability of the integration process, the spectral radius of the matrix must be not greater than 1. For instance, one can consider the polynomial $r(s) = s^n$ which is equivalent to the choice $l_j \sim (0, \dots, 0, 1)^T$. However, the substitution of these values into Eq. (31) results in unstable iteration as the spectral radius of the matrix $\hat{M}_0$ becomes greater than one.

One can check by the direct substitution, that the spectrum of $M(\omega)$ contains eigenvalue $\{0\}$ corresponding to the eigenvector l^j for any ω . The distribution of remaining part of the spectrum depends on ω and l^j . It is expected (Hindmarsh, 1983; Radhakrishnan and Hindmarsh, 1993) that the integration process is stable for an arbitrary large ω . Therefore, it is required that the spectrum of matrix $M(\omega)$ for $\omega \rightarrow \infty$,

$$M_j^k(\infty) = P_m^k \left(\delta_j^m - \frac{1}{r} \tau_j l^m \right). \quad (32)$$

is always located within the unit circle. Remind here $\tau_j = (1, 1, 1, \dots, 1)^T$ and $r = l^j \tau_j$. Specifically, it is required that $M(\infty)$ has all zero eigenvalues, which is equivalent to that the matrix is nilpotent and $M^q(\infty) = 0$. The derivation of the coefficients l^j from the nilpotent property can be found in Hindmarsh (1983). Table 1 reproduces these coefficients for orders q from 1 to 6, as well as the parameters $r = l_1 + \dots + l_2$ and $\dot{r} = l_1 + 2l_2 + \dots + 6l_6$ used for calculation of α and β_i . It can be verified that the matrix $M(\infty)$ for these coefficients has all zero eigenvalues by computing them numerically. However, the eigenvalues of this matrix are very sensitive to round-off errors, so they should be calculated with precision much higher than double.

The working examples of the implementation of the BDF method in Nordsieck's formulation are presented in journal papers (Nordsieck,

Table 1
Coefficients of BDF for different orders n .

n	l_0	l_1	l_2	l_3	l_4	l_5	l_6	r	$\dot{r}$
1	0	1	–	–	–	–	–	1	1
2	0	1	1	–	–	–	–	2	3
3	0	2	3	1	–	–	–	6	11
4	0	6	11	6	1	–	–	24	50
5	0	24	50	35	10	1	–	120	274
6	0	120	274	225	85	15	1	720	1764

1962; Kulikov, 2009; Cherezov et al., 2023; Zimin and Cherezov, 2017) and LSODE manual (Radhakrishnan and Hindmarsh, 1993). The algorithm can be applied to the neutron kinetics system given by Eqs. (15) and (16), where vector y_i consists of the flux and precursor components $\Phi_{ig}(t)$ and $R_m^j(t)$, and f_i represents a linear function. However, the method can be also applied to a general case, where f_i represents the coupled system of reactor physics equations, including neutron kinetics, thermal-hydraulics, and thermal-mechanics.

2.4. In-step multigroup kinetics

BDF method applied to the neutron kinetics equations represented by Eq. (15) and (16) implies the solution of a linear system of equations given by Eq. (25) at every time step. In the expanded form the in-step equation reads

$$V_g^{ij} (\Phi_{ig} - \hat{\Phi}_{ig}) = -L_g^{ij} \Phi_{ig}^s + S_{gg'}^{ij} \Phi_{ig}^{s'} + (1 - \beta) \chi_g F_g^{ij} \Phi_{ig}^{s'} + \chi_g' \lambda_m^m R_m^j, \quad (33)$$

$$R_m^j - \hat{R}_m^j = h \beta_m F_m^{ij} \Phi_{ig}^{s'} - h \lambda_m R_m^j, \quad (34)$$

where h is the step size multiplied by the parameter that depends on the BDF order. Matrix $V_g^{ij} = (h v_g)^{-1} \epsilon^{ij}$ is associated with the neutron velocity, and vectors $\hat{\Phi}_{ig}^j$ and $\hat{R}_m^j$ are the neutron and precursor sources acquired at the previous time step and computed according to the right-hand side of Eq. (25). At the beginning of the integration process, these vectors are computed from the steady-state reactor problem (Cherezov et al., 2024). A semi-analytical approach can be used for the numerical integration of the precursor equations. In this method, the time-dependent neutron flux is approximated by a linear function of time, and an analytical formula is derived based on the time-dependent exponents. An example of implementing this semi-analytical approach can be found in Avvakumov et al. (2016).

Substitute term R_m^j of the second equation into the first one. Then, the in-step system of multigroup equations reads

$$(V_g^{ij} + L_g^{ij}) \Phi_{ig} = S_{gg'}^{ij} \Phi_{ig}^{s'} + \eta_g F_g^{ij} \Phi_{ig}^{s'} + \hat{X}_g^j, \quad (35)$$

where $\eta_g = (1 - \beta) \chi_g + \lambda_m h / (1 + \lambda_m h) \beta^m \chi_g^m$ and summation over m is assumed as usual. The source vector on the right hand side is calculated as follows

$$\hat{X}_g^j = V_g^{ij} \hat{\Phi}_{ig} + \chi_g' \alpha_m \hat{R}_m^j. \quad (36)$$

Once neutron flux Φ_{ig} has been calculated, the vector of precursors is calculated straight away from Eq. (34) without additional iterations.

The system of Eq. (35) can be represented in matrix–vector form using a large sparse non-symmetric matrix with energy group blocks. There are two methods to solve this multi-group system. The first method is the GS group-by-group iterations, using a preconditioned CG method to solve the in-group diffusion equations. The second method involves solving the entire multi-group system with a preconditioned Krylov method, such as Bi Conjugate Gradient (BiCG). The following subsections provide detailed discussions on these two approaches.

2.4.1. GS/CG method

The block GS method (Hageman and Young, 1981; Saad, 2003) used in FEMCORE is based on the separation of the diagonal block matrices $H_g^{ij} = V_g^{ij} + L_g^{ij}$ from the scattering and fission block matrices $Q_{gg'}^{ij} = S_{gg'}^{ij} + \eta_g F_g^{ij}$. A GS iteration for an energy group g reads

$$\Phi_g^{k*} = \tilde{H}_{Jg}^k (Q_{gg'}^{ij} \Phi_{ig}^{s'} + \hat{X}_g^j), \quad (37)$$

assuming that there exists an inverse matrix $\tilde{H}_{Jg}^k$ to H_g^{ij} for each energy group g . In this equation, indices i, j and g' are summed, while indices k and g are free.

In the algorithm, each GS iteration performs in two steps. In the first step, the combined scattering and fission source $Q_{gg'}^{ij} \Phi_{ig}^{s'} + \hat{X}_g^j$ is calculated using the neutron flux $\Phi_{ig}^{s'}$ from the previous iteration. In the second step, the new neutron flux Φ_g^{i*} is computed by formally multiplying the combined source by matrix $\tilde{H}_{Jg}^k$. These steps are repeated for each energy group g , usually from fast to thermal, until the convergence of the neutron flux for all groups is achieved.

The convergence criteria of the GS iteration (Saad, 2003) is the strict diagonal dominance of the block matrix $H_g^{ij} - Q_{gg'}^{ij}$, that is the prevalence of time evolution and neutron loss over fission and scattering processes. Then, the convergence rate is equal to the spectral radius of the iteration matrix $G_{gg'}^{ki}$ understood as the entire multi-group matrix, and equal to

$$G_{gg'}^{ki} = \tilde{H}_{Jg}^k Q_{gg'}^{ij}, \quad (38)$$

with the summation over index j . The dominance of H_g^{ij} over $Q_{gg'}^{ij}$ is satisfied for a small step size $h \ll \tau_g$, where the prompt neutron lifetime $\tau_g \sim 1/(v_g \Sigma_g)$, and matrix V_g^{ij} is relatively large. If the time step h gets much larger, the term V_g^{ij} becomes too small, and the GS method may slow down or diverge in the worst case. However, the H_g^{ij} dominance is still expected for a large step size h if the reactor is subcritical, where the neutron loss term L_g^{ij} significantly exceeds the scattering plus fission term $Q_{gg'}^{ij}$.

One approach to improving convergence of the GS method is to utilize an acceleration method that corrects the solution vectors at each GS iteration based on extrapolation from previous iterations. Chebyshev extrapolation (Hageman and Young, 1981; Hageman and Yasinsky, 1969) is the acceleration method commonly applied for the nodal methods. The drawback of the Chebyshev method is that it requires the estimation of the minimum and maximum eigenvalues of the iterated matrix, which potentially leads to a dramatic slowdown in convergence if the estimation is not enough accurate. Alternatively, one can employ potentially more efficient Anderson's method that does not depend on the knowledge of the matrix eigenvalue spectrum.

Matrix H_g^{ij} of a given energy group g is a symmetric positive-definite matrix since it approximates a self-adjoint operator in a Hilbert space. Therefore, the linear system of equations associated with this matrix can be solved by the Krylov methods, such as CG or QMR. However, the condition number of this matrix is typically very large and tends to increase with the FEM degree. Therefore, it is necessary to apply a preconditioning matrix to reduce the condition number and the number of Krylov iterations. There are many preconditioning methods that can be used for neutron transport and/or diffusion equations, including classical Jacobi, SSOR, and ILUT methods (Bru et al., 2002), domain decomposition (Vidal-Ferràndiz et al., 2017) and AMG (Ragusa, 2015). In this study, we utilized a simple yet effective ILU(0) factorization where the lower and upper triangular matrices L and U retain the same sparsity pattern as the matrix before factorization. In our implementation, the ILU(0) factorization of matrices H_g^{ij} is computed for all energy groups g only once, at the first time step, with step size h set to one. Afterwards, the same preconditioning matrices are applied to all subsequent time steps.

2.4.2. BiCG method

Alternatively, the multi-group system of in-step Eq. (35) can be solved as a whole system using a Krylov subspace iteration method adapted to the non-symmetric matrices, such as biconjugate gradient stabilized method (BiCG). This algorithm is characterized by a higher convergence rate over iterations than the GS/CG method. However, each iteration involves two preconditioning steps and vector-by-matrix multiplications, that decreases overall solver performance. The preconditioning procedure can be implemented in many different ways. In this study, we employed a GS/CG iteration as a preconditioning step. The iteration involves the solution of diffusion equations for each group g by the ILU(0)-preconditioned CG method. The stopping criteria RTOL for CG and BiCG was set to 10^{-1} and 10^{-6} , respectively.

2.5. Anderson method

Anderson's method takes the vectors of the previous iterations and combines them in an optimal way to achieve faster convergence. It has been shown (Eyert, 1996) that the Anderson's method is equivalent to a quasi-Newton–Raphson method updating an approximate inverse Jacobi matrix at each iteration in such a way that the distance between the Jacobian and its predecessor is required to be minimal. This method, however, does not need nor inversion or storage of the full Jacobi matrix, and can be written in a quite simple representation as a linear form of residual vectors.

Consider the problem of finding a root vector x_k^* of the non-linear vector function $f_k = f_k(x_k)$, which is the solution of the equation $f_k(x_k^*) = 0$, where index k denotes all vector components according to the notation of Appendix A. This function refers to Eq. (37) for the GS method or Eq. (25) for the BDF method. Let f_k^i be the evaluations of function f_k at vectors x_k^i of the previous m iterations, with x_k^1 and x_k^m being the newest and the oldest vectors, respectively. The next vector x_k^0 in the iteration sequence is estimated as follows

$$x_k^0 = \beta_i (x_k^i + \alpha f_k^i), \quad (39)$$

summing over index $i = 1, \dots, m$. The choice of the mixing parameter α can significantly impact the convergence properties of the method. The analysis of the method convergence for various α is presented in Fang and Saad (2009), Hamilton et al. (2016). In this study we assume the simple mixing scheme with $\alpha = 0.5$.

At each iteration of the algorithm, the constrained optimization problem subject to vector β_i needs to be solved,

$$\begin{cases} \beta_i = \arg \min (\beta_i \beta^j f_k^i f_j^k), \\ \beta^j e_j = 1, \end{cases} \quad (40)$$

summing over indices i, j and k . Here, f_k^i denotes the transposition of the non-symmetric array f_k^i and vector $e_j \sim (1, 1, \dots, 1)^T$. This problem minimizes the discrepancies of function f_k from 0 at m previous iterations. Applying the variational analysis to the Lagrangian of this problem,

$$\mathcal{L}(\beta, \lambda) = \beta_i \beta^j f_k^i f_j^k + 2\lambda (\beta^j e_j - 1), \quad (41)$$

where the Lagrangian multiplier λ is introduced to incorporate the constraints, we derive the following equations

$$\begin{cases} f_k^i f_j^k \beta_i = \lambda e_j, \\ \beta_1 + \dots + \beta_m = 1, \end{cases} \quad (42)$$

from which vector β_i and multiplier λ can be uniquely identified. These equations can be solved using a linear system solution method, e.g. LU factorization. However, solving this equation is complicated because the matrix $f_k^i f_j^k$ becomes ill-conditioned near a root where $f_k^0 \approx f_k^1$. Therefore, regularization of the matrix is required. To achieve this, we

multiply the diagonal elements of the matrix by a coefficient ω close to one, so that the system of equations reads

$$(1 + \omega \delta_j^i) f_k^i f_j^k \beta_i = \lambda e_j, \quad (43)$$

where δ_j^i is the Kronecker's symbol. Thus, the system of equations is always non-singular and can be solved using a direct method, such as the LU factorization. An alternative approach involves using QR or SVD factorization applied to the rectangle matrix f_k^i ; this approach was investigated in Fang and Saad (2009). However, in our numerical tests, diagonal regularization showed slightly better results. Therefore, FEMCORE implements the solution of Eq. (42) using regularization method with coefficient $\omega = 10^{-3}$. The choice of the number of vectors m depends on the simulation. Our previous analysis (Cherezov et al., 2024) showed that $m = 10$ is optimal for steady-state problems, so we used this value throughout this work. The concise derivation of the Anderson's method is presented in Fang and Saad (2009), Eyert (1996).

2.6. Automatic control

The integration begins with the first order BDF and a user-provided step size. As the integration progresses, both the step size h and the method order q evolve to optimal values selected in such a way that the local truncation error $e_h < 1$, which is estimated as follows

$$e_h = \frac{\|x_h^p - x_h^c\|_2}{\|a_{tol} + r_{tol} \cdot x_h^c\|_2}, \quad (44)$$

where numerator is the difference between the predicted x_h^p and corrected solution x_h^c calculated at the previous time step, r_{tol} and a_{tol} are the relative and absolute tolerances. Then, the new step size h' is calculated from the following equation

$$\frac{h}{h'} = C_1 (D_q e_h + C_2)^{\frac{1}{q+1}}, \quad (45)$$

where h is the previous step size, q is the method order, D_q is the parameter that depends on the method order, and $C_1 = 1.2 \dots 1.4$ and $C_2 = 10^{-6}$ are the safety coefficients. At every time step, the new step size is calculated three times for orders q , $q-1$ and $q+1$. Then, the largest step is selected with respectful change of the method order. The details on the calculation of the parameters one can find in the LSODE's manual (Radhakrishnan and Hindmarsh, 1993).

LTE is estimated after each time step, and if it exceeds 1, the time step is rejected and the process repeats with a reduced step size by factor two. Time step rejection can occur due to inconsistency between the order of the integration scheme and the smoothness of the solution. Solution smoothness may be compromised due to insufficient number of iterations of the in-step equation or abrupt changes in compositions, associated, for example, with the movement of a control rod. Overall, the large number of rejected time steps indicates the convergence issues and the need in more inner iterations to solve the instep equation with higher accuracy.

Note that there are numerous methods for automated selection of a step size developed for integration of neutron kinetics equations using step-doubling method (Cherezov et al., 2020; Sutton and Aviles, 1996), inverse period of a particular state variable (Sutton and Aviles, 1996), local truncation errors of a backward difference formula (Boffie and Pounders, 2018; Schmid and Wagner, 1995; Avvakumov et al., 2019), modal difference error estimator (Carreño et al., 2021), proportional–integral–derivative controller (Jaradat and Sik Yang, 2023). In this study, we employed the LSODE's step control method (Radhakrishnan and Hindmarsh, 1993), which permits adjustments to both the step size and the approximation order. The BDF algorithm used in FEMCORE is like the one in LSODE but adapted to solve the in-step equation using Anderson's method. It employs five times tighter convergence criteria for Eq. (25) to reduce step rejections, as in-step iterations with Anderson's method are significantly more costly than the Newton–Raphson iterations used by LSODE.

2.7. FEMCORE

FEMCORE is a numerical tool that solves the steady-state and transient multi-group diffusion equation using the variety of methods described above. The input for the code includes arrays ϵ_e^{ij} , δ_e^{ij} and σ_e^{ij} representing the reactor core geometry, and the material compositions distributed over finite elements in form of arrays D_g^e , Σ_g^e , S_g^e , and F_g^e . This general input format enables the application of the code to discretized multi-group equations using either finite element or finite volume methods. FEMCORE transient simulation algorithm is presented in [Appendix C](#).

The primary computational costs of the code are associated with matrix–vector multiplication, preconditioning procedures, and matrix assembly. To enhance efficiency, matrices are stored in the Compressed Sparse Row (CSR) format. The CSR format stores a sparse matrix using three one-dimensional arrays: the non-zero values (VAL), the column indices (COL) of those values, and the indices (PTR) of the column indices array where the given row starts. The arrays COL and PTR are calculated at the beginning of code execution based on the input geometric arrays. Subsequently, the values of the matrices are updated when cross-section parameters change. The built-in matrix-by-vector and preconditioning procedures can be replaced by a third-party back-end, optimized for specific hardware.

3. Numerical results

In this study, we evaluate the performance of our algorithm against three transient benchmarks: SKINATH, TWIGL, and LRA-2D. The Simple KINetics and Thermal Hydraulics (SKINATH) benchmark employs a point-kinetics model with temperature feedback, and we used it to verify the BDF method in comparison to LSODE. The TWIGL benchmark represents a highly simplified two-dimensional model of a PWR with no feedbacks, designed to simulate the power response due to the ejection of a control rod. The LRA-2D benchmark is a two-dimensional model of a BWR with temperature adiabatic feedback, simulating power transients resulting from a control rod fall accident. Through these benchmarks, we aim to illustrate the performance of the BDF and Anderson methods.

3.1. SKINATH benchmark problem

Consider a Simple Kinetics and Thermal Hydraulics (SKINATH) model applied to an air-cooled homogeneous cylindrical critical system for investigation of low power criticality accidents. SKINATH model comprises the m -group neutron kinetics that reads

$$\begin{cases} \dot{N} = (\rho - \beta)N/\Lambda + \sum_{i=1}^m \lambda_i C_i, \\ \dot{C}_i = \beta_i N/\Lambda - \lambda_i C_i, \quad i = 1, \dots, m, \end{cases} \quad (46)$$

where N is the reactor power, C_i is the precursor concentration for group i , ρ is the reactivity, β_i is the yield from fission of precursors for group i , λ_i is the precursor decay constant for group i , β is the total yield, Λ is the neutron generation time. The reactivity is given by

$$\rho(t) = \rho_0 + \beta_T(T(t) - T_c) \quad (47)$$

where $\rho_0 = 0.043\beta$ is the initial reactivity and $\beta_T = -0.00306\beta/^\circ\text{C}$ is the temperature reactivity coefficient. The temperature is described by a simple thermal-hydraulic model with the parameters in Table 6a of [Ganapol \(2013\)](#). The energy balance in terms of an average temperature is given by

$$\dot{T} = \frac{1}{C_p} \left(N(t) - ahd^{\frac{3}{4}} \left[1 - \frac{T_c}{T(t)} \right]^{\frac{1}{4}} (T(t) - T_c) \right). \quad (48)$$

Here $h = 23$ cm and $d = 20$ cm are the system height and diameter respectively, $T_c = 20^\circ\text{C}$ is the initial temperature, $a = 17.52$ is a physical

constant for air at STP, $C_p = 1.8 \cdot 10^{-6} \pi h d^2 / 4 \text{ Erg}/^\circ\text{C}$ is the system heat capacity. The initial power $N_0 = 0.01 W$. The power pulse transient resulting from exceeding reactivity ρ_0 is simulated for a duration of 250 min.

[Fig. 4](#) displays the variation in neutron density alongside the evolution of step size, method order, and the number of iterations required for solving the in-step non-linear equation. The cross marks on the graphic of the time step history indicate the rejected time steps, for which the local truncation error exceeds tolerance.

[Fig. 5](#) shows the work-error graphics computed by LSODE and FEMCORE using Anderson's method for solving the in-step equation. The error is the difference between reference neutron density at the end of simulation time, and the work is measured in the number of evaluations of the right-hand side function $f(t, x)$ called during the iteration of Eq. (25).

Significant discrepancies are observed in the integration history produced by FEMCORE and LSODE, attributed to differences in their in-step equation solvers. LSODE utilizes the Newton iteration method for solving the in-step equation, with the Jacobian matrix J estimated as a full matrix in dense format using the finite difference quotient method. Note, for large systems of equations, the Jacobian matrix can be computed as a banded matrix, preserving non-zero elements on a few bands parallel to the diagonal. Subsequently, the linear system $x - hJx = b$ is factorized into lower and upper triangular matrices L and U , followed by back-solving to obtain $x = U^{-1}L^{-1}b$. LSODE attempts to reuse the same matrices L and U over multiple steps to reduce computational costs, employing the same Jacobian matrix for a maximum of 20 steps and updating it when the implicit equation fails to converge for a maximum of 3 iterations or when three or more error test failures occur on any step. Additionally, matrices L and U are updated every time J is reevaluated, and if the step size h changes by more than 30.

The maximum number of in-step iterations is set equal to three as “a larger number increases the computation costs without a corresponding increase in the probability of successful convergence” ([Radhakrishnan and Hindmarsh, 1993](#)). Should the non-linear in-step equation fail to converge within three iterations, the code updates the Jacobian matrix and reduces the current step size by a factor of four. This approach has dual effects on the computation costs, which is associated mostly with the number of evaluations of the non-linear function. Firstly, it increases the likelihood of step rejection, and, thereby potentially increasing the number of evaluations. Secondly, it decreases the number of evaluations, as the same Jacobian matrix is likely to be reused for several successive steps. Therefore, while the average number of iterations may fluctuate frequently, it remains relatively low, as demonstrated in one of the numerical examples.

In FEMCORE, the Anderson's mixing vectors have to be updated on every time step, hence, it is more beneficial to increase the probability of successful convergence. Therefore, the code rejects a time step if the number of Anderson's iterations exceeds ten. This increases the number of iterations if the time step fails to converge, however, the time step becomes more stable and overall number of iterations is not very high.

3.2. TWIGL-2D benchmark problem

3.2.1. Description

The TWIGL benchmark problem originally described in the paper of [Hageman and Yasinsky \(1969\)](#) represents a two-dimensional model of a Pressurized Water Reactor (PWR). The reactor core consists of one blanket and two seed fuel regions. The zero-flux boundary conditions are employed for the core without reflector. The benchmark employs two energy group and one delayed neutron family. The initial reactor state is critical, and the positive reactivity is inserted in one of the seed fuel regions by decreasing the thermal absorption cross-section. In this study we considered the ramp perturbation when the thermal absorption cross-section is reduced linearly. The total amount of reactivity inserted is $\Delta\rho = 384 \text{ pcm}$ that is less than the effective delayed neutron fraction $\beta_{eff} = 750 \text{ pcm}$ and results in a delayed super-critical transient. The detailed description of the benchmark including layout and compositions can be found in [Kotchoubey \(2015\)](#), [Gehin \(1992\)](#).

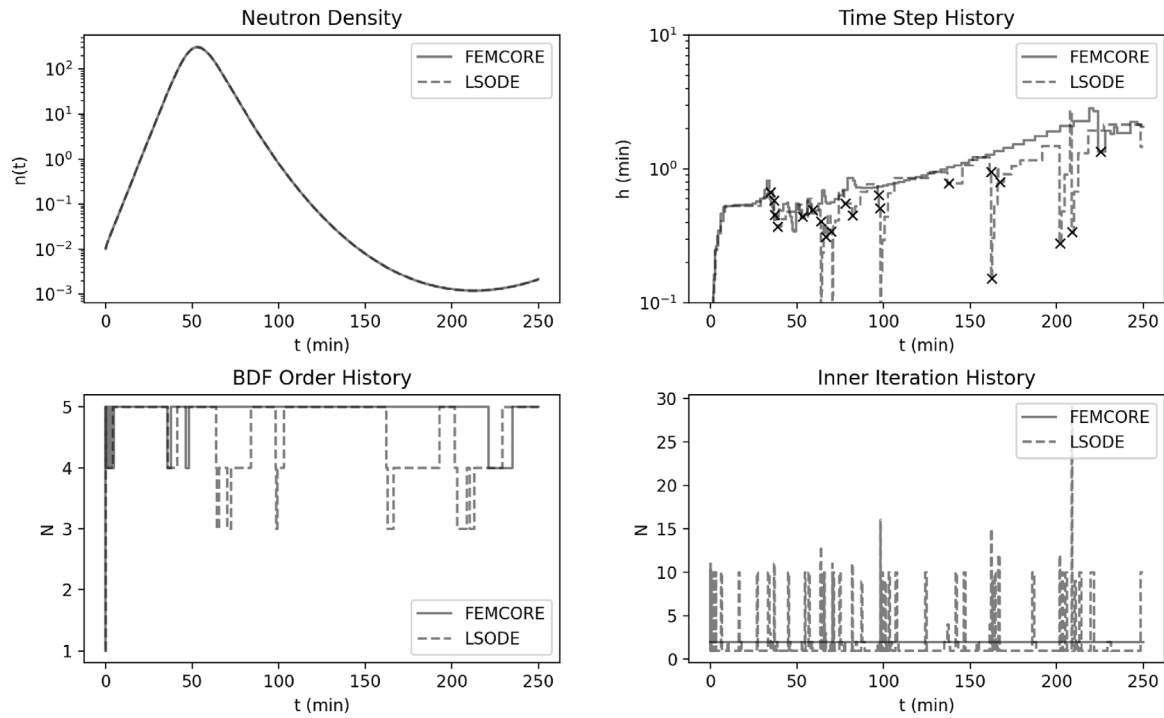


Fig. 4. The SKINATH benchmark performance, RTOL=1E-6.

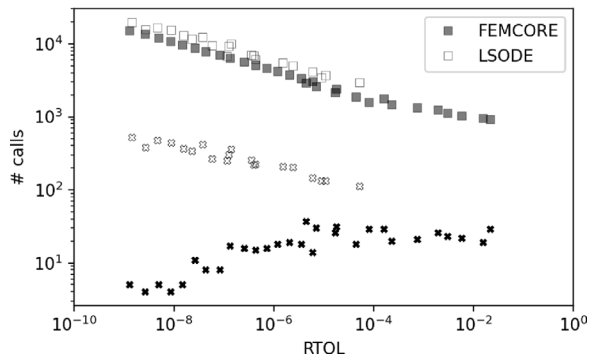


Fig. 5. Work-error graphic for the SKINATH benchmark showing the number of time steps (□) and rejected steps (*).

3.2.2. Transient analysis

The computed effective multiplication factor $k_{eff} = 0.913211$ matches exactly with most of published results of this benchmark. To achieve reactor criticality, the fission production cross-sections are divided by k_{eff} . The time-dependent power, computed using 4th order FEM and BDF relative tolerance RTOL=10⁻⁶, is presented in Table 2. The power observes small error compared to the two-group polynomial nodal diffusion code CUBBOX (Langenbuch et al., 1996), multi-group analytic nodal method code POLCA-T (Kotchoubey, 2015), and functional expansion nodal method code ANTS (Rintala and Lauranto, 2023). Fig. 6 shows the distribution of neutrons for fast and thermal energy groups at the end of simulation time.

3.2.3. BDF performance

Fig. 7 illustrates the performance of the BDF method, showing the history of reactor power rate P , time step size h , integration order q , and the cumulative number of matrix-vector multiplications (MUL) during the simulation. The graphics were computed using BDF methods of 1st, 2nd, and 5th orders, referred to as BDF-1, BDF-2, and BDF-5, respectively, with maximum orders restricted to 1, 2, and 5. Cross

Table 2

TWIGL-2D results.

Time	FEMCORE	CUBBOX	POLCA-T	ANTS
0.0 s	1.000	1.000	1.000	1.000
0.1 s	1.309	1.321	1.308	1.309
0.2 s	1.957	1.985	1.961	1.961
0.3 s	2.076	2.074	2.076	2.075
0.4 s	2.094	2.092	2.093	2.093
0.5 s	2.111	2.109	2.111	2.110

marks on the time step history graph denote rejected time steps. In all cases, the integration order starts from 1 and gradually increases to the maximum order. The initial time step was set to $h = 10^{-5}$ s. The drop of step size at time 0.2 s is associated with the discontinuity of the first order derivative of the thermal cross-section over time. This discontinuity causes a wrong prediction of the local relative error and a sequence of step rejections. As shown in the cumulative MUL graphic, the BDF-5 method significantly outperforms the 1st and 2nd order BDF schemes.

3.2.4. Anderson performance

The problem has been solved several times for each FEM degree varies from 1 to 8 using the Anderson's, Chebyshev, and Aitken acceleration methods, as well as the BiCG solver. The performance of the methods is illustrated in Table 3. In this table, DOF represents the number of degrees-of-freedom for one energy group, STP is the count of successful time steps, REJ is the count of rejected time steps, GS/An is the count of GS iterations enhanced by the Anderson's method, MUL is the number of matrix-vector multiplications, and CPU is the time spent on the problem. Additionally, the last three columns display the performance of the GS iterations enhanced by Chebyshev (GS/Ch) and Aitken (GS/At) method, and full multi-group matrix solver (BiCG) in comparison to the default GS/An method. These columns depict the ratios of the number of matrix-vector multiplications of the Chebyshev, Aitken, and BiCG methods to those of Anderson's method.

As observed, the number of iterations (MUL) increases with the degree of FEM, while the number of GS iterations remains relatively

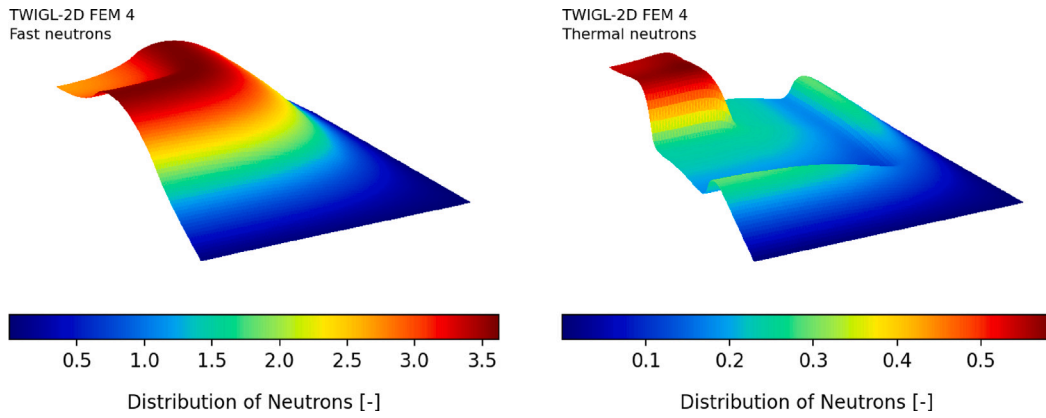


Fig. 6. TWIGL-2D neutron flux.

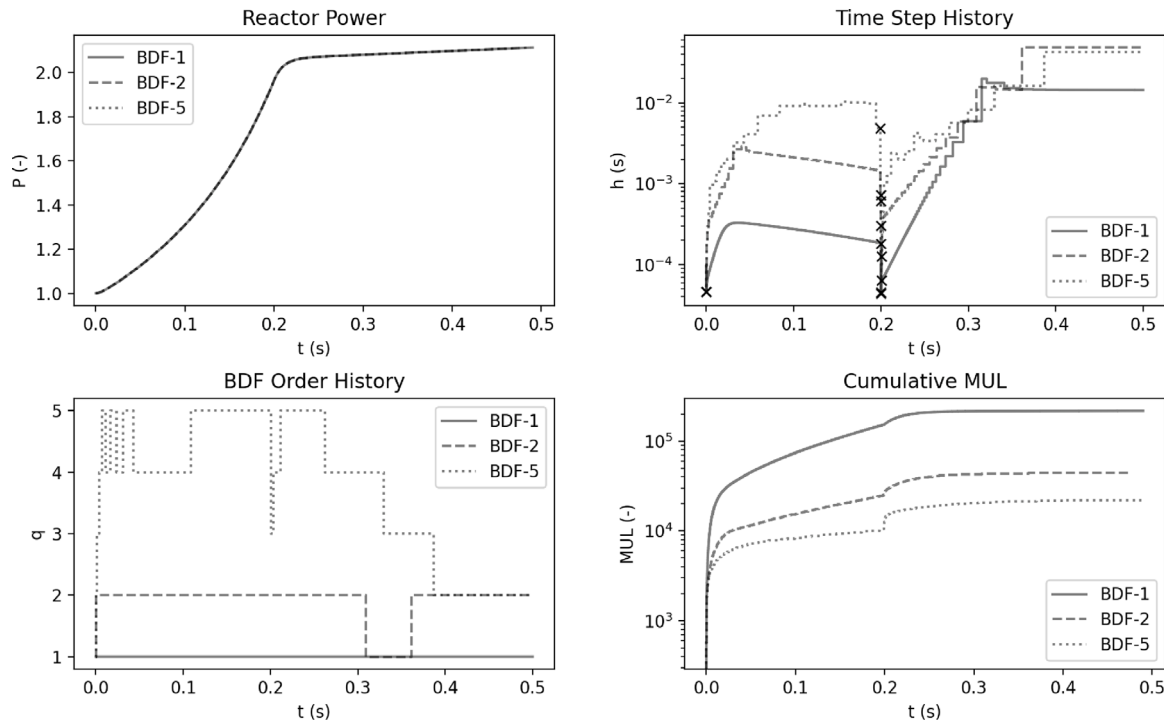


Fig. 7. Performance of BDF for TWIGL-2D.

constant. This behavior is logical, considering that the condition number of the matrix grows with the polynomial order (Wang and Ragusa, 2009), and the employed preconditioning method ILU(0) cannot fully compensate this effect. The number of time steps ranges from 134 to 146, with a relatively small number of rejected time steps. Rejected time steps are primarily associated with moment 0.2 s when the thermal cross-section stops change. The performance of the Anderson's method is significantly better than for other methods.

Fig. 8 presents the number of time steps (A), number of GS iterations (B) and their ratio (C) for the TWIGL-2D benchmark solved using various relative tolerances RTOL of the BDF scheme. The graphics illustrate the performance of the GS method without acceleration (GS) and the performance of the GS method accelerated by Chebyshev method (GS/C) and Anderson's method (GS/A- m), where $m = 2, 4, 6$ is the method's depth. It is evident from the graphics that $m = 4$ is sufficient for the Anderson's method since further increase in depth does not improve in the performance.

3.3. LRA-2D BWR benchmark problem

The LRA benchmark problem (Finnemann, 1977) represents a model of a Boiling Water Reactor (BWR) comprising 312 fuel assemblies surrounded by a water reflector. This model is characterized by two energy groups and two delayed neutron precursors. The benchmark is designed to simulate an asymmetric peripheral control rod drop accident, resulting in a prompt supercritical reactivity excursion. Its computational complexity arises from severe flux tilts and gradients, demanding a highly accurate spatial neutronics model. Additionally, the power excursion of approximately ten orders of magnitude introduces complexity to the time integration process. Over the years, the benchmark has been widely utilized for the verification of nodal methods (Smith, 1979; Zimin and Ninokata, 1998; Rintala and Lauranto, 2023), finite element methods (Senecal and Ji, 2018; Wang et al., 2021), and analytical methods (Aboanber and Hamada, 2009).

Table 3
Performance of TWIGL-2D.

FEM		BDF Steps		Num. of iterations		CPU	MUL ratio to GS/An		
Order	DOF	STP	REJ	GS/An	MUL	s	GS/Ch	GS/At	BiCG
1	121	135	5	1604	18 292	0.2	5.2	6.8	2.0
2	441	146	5	1712	22 174	0.6	4.9	6.6	1.8
3	961	146	8	1744	25 223	1.9	5.5	8.5	1.5
4	1681	134	2	1554	25 193	4.1	6.3	9.3	1.6
5	2601	137	6	1619	28 710	9.4	5.0	8.6	1.5
6	3721	138	6	1612	30 589	18.6	4.9	7.7	1.4
7	5041	134	5	1581	32 003	34.9	5.1	8.4	1.3
8	6561	134	7	1630	35 015	65.5	5.1	8.2	1.3

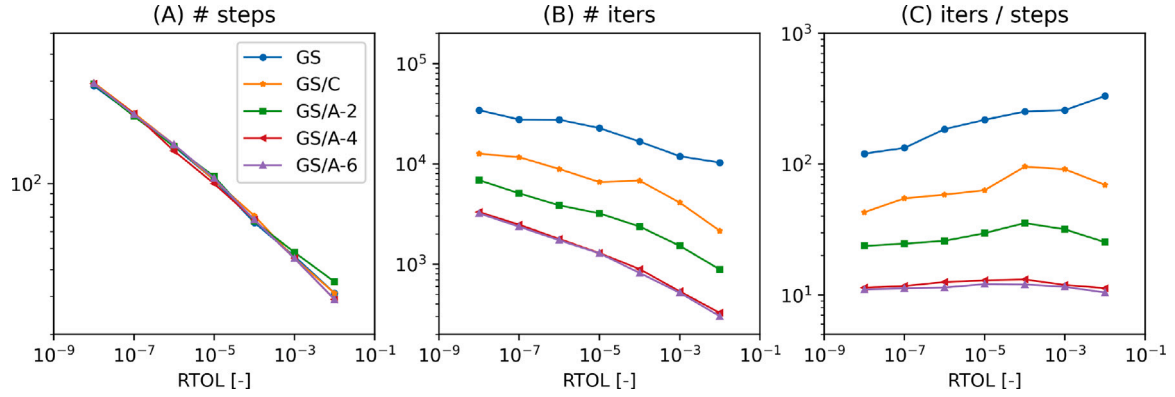


Fig. 8. Number of steps and GS iterations.

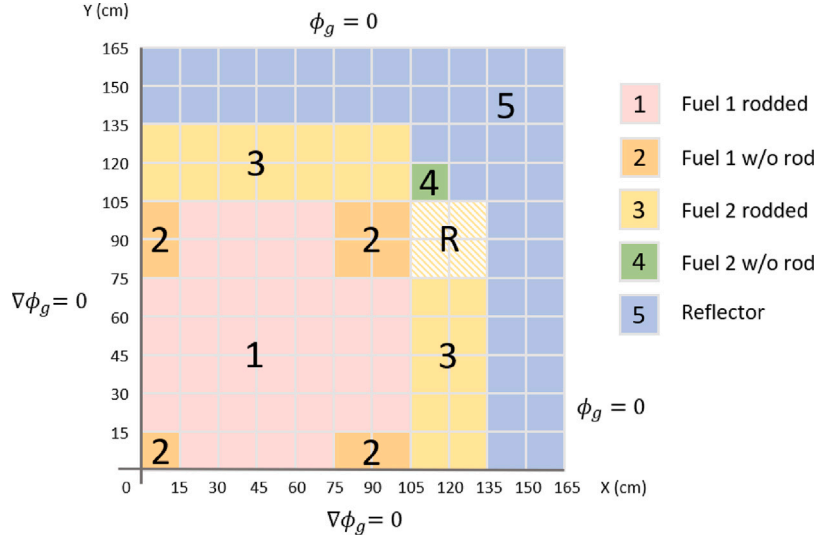


Fig. 9. LRA BWR layout.

3.3.1. Description

The benchmark offers the flexibility of being modeled in either two or three dimensions. This study is focusing on the two-dimensional quarter-core version with the axial buckling $B^2 = 10^{-4} \text{ cm}^{-2}$ used in both energy groups to address neutron leakage in axial directions. The layout and assignment of the regions are shown in Fig. 9. Table 4 presents the two-group homogenized cross-sections of the compositions. Other parameters of the neutronic models are: the mean number of neutrons per fission $\nu = 2.43$, the fast and thermal neutron velocities $v_1 = 3 \cdot 10^7$ and $v_2 = 3 \cdot 10^5 \text{ cm/s}$. The delayed neutron precursors are characterized by the delayed neutron fractions $\beta_1 = 0.0054$ and $\beta_2 = 0.001087$, decay constants $\lambda_1 = 0.0654 \text{ s}^{-1}$ and $\lambda_2 = 1.35 \text{ s}^{-1}$.

Feedback mechanisms on neutronics are incorporated through adiabatic heat-up and space-dependent Doppler effect. Other feedback

effects like moderator boiling and thermal expansion are not considered by the benchmark to limit problem's complexity. The adiabatic heat-up model reads

$$\frac{\partial T}{\partial t} = \alpha (\Sigma_{f1}\phi_1 + \Sigma_{f2}\phi_2), \quad (49)$$

where $\alpha = 3.83 \cdot 10^{-11} \text{ K} \cdot \text{cm}^3$ is the energy conversion factor divided by the volumetric heat capacity. The Doppler effect for the first-group absorption cross-sections is calculated as follows

$$\Sigma_{a1}(t, x) = \Sigma_{a1}(0, x) \left(1 + \gamma \left(\sqrt{T(t, x)} - \sqrt{T_0} \right) \right), \quad (50)$$

with the feedback constant $\gamma = 3.034 \cdot 10^{-3} \text{ 1/K}^{0.5}$ and initial temperature $T_0 = 300 \text{ K}$. It should be noted, that in our simulations we used the element-wise representation of the temperature T , that is the average temperature is considered for each finite element. This approximation

Table 4
Cross-sections for the LRA BWR benchmark.

Region	D_1	D_2	Σ_{a1}	Σ_{a2}	Σ_{d12}	$\nu\Sigma_{f1}$	$\nu\Sigma_{f2}$
1	1.255	2.110E-1	8.252E-3	1.0030E-1	2.533E-2	4.602E-3	1.091E-1
2	1.268	1.902E-1	7.181E-3	7.0470E-2	2.767E-2	4.609E-3	8.675E-2
3	1.259	2.091E-1	8.002E-3	8.3440E-2	2.617E-2	4.663E-3	1.021E-1
4	1.259	2.091E-1	8.002E-3	7.3324E-2	2.617E-2	4.663E-3	1.021E-1
5	1.257	1.592E-1	6.034E-3	1.9110E-1	4.754E-2	0	0

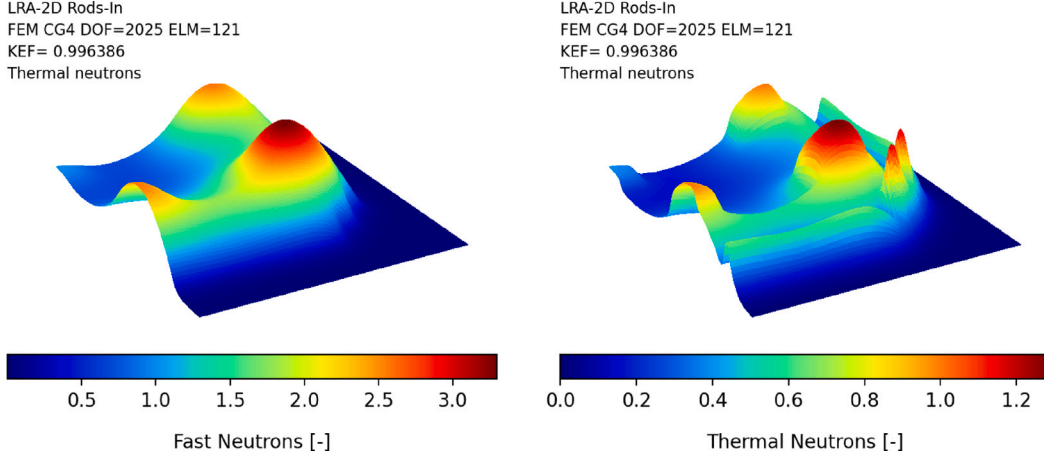


Fig. 10. LRA-2D neutron flux at 0s.

introduces some inaccuracy into the model, which is not large and was studied previously (Smith, 1979). The average reactor core power $\bar{P}$ and fuel temperature $\bar{T}$ are calculated as follows

$$\bar{P}(t) = \frac{\epsilon}{V_{core}} \int_{V_{core}} (\Sigma_{f1}\phi_1 + \Sigma_{f2}\phi_2) dV, \quad (51)$$

$$\bar{T}(t) = \frac{1}{V_{core}} \int_{V_{core}} T(t, x) dV, \quad (52)$$

where $\epsilon = 3.204 \cdot 10^{-11}$ J/fission is the energy conversion factor, $V_{core} = 17750 \text{ cm}^3$ is the reactor core volume comprised of the all regions except the reflector region 5.

3.3.2. Steady-state analysis

The steady-state problem for the initial state of LRA 2D is solved using the eigenvalue algorithm (Cherezov et al., 2024), which is based on the power iteration method accelerated by Anderson's method. The fission source equation is solved group-by-group using GS iterations, also accelerated by Anderson's method. The within-group diffusion equation is solved using the CG method with ILU(0) preconditioning. The steady-state distribution of neutrons is calculated without temperature feedback at $T_0 = 300 \text{ K}$, and then used as the initial vector for the transient analysis. A relative tolerance $r_{tol} = 10^{-9}$ is set for the eigenvalue solver, ensuring a small error in the initial values of the time-dependent problem.

Figs. 10 illustrates the distribution of neutrons for the fast and thermal energy groups for the rod-in case. The both distributions are characterized by their maximum values located in region 2 at the reactor corner. The distribution of thermal neutrons exhibits large gradients with two distinct peaks in the reflector due to the thermalization effect. The corresponding multiplication factor exhibits an excellent agreement with the reference value computed by the nodal code CUBBOX (Langenbuch et al., 1996) with the discrepancy within 5 pcm, as detailed in Table 6.

3.3.3. Transient analysis

The transient was computed starting from the steady-state distribution of neutrons scaled to an average power $\bar{P}_0 = 10^{-6} \text{ W/cm}^3$. The initial temperature of reactor is uniform, $T(0, x) = T_0$, and the

Table 5
Power and temperature peaks of LRA 2D.

FEM	t_1	P_1	t_2	P_2	T_3	$\delta P_3, \%$	$\delta T_3, \%$
1	1.352	5820	2.002	854	109.9	13.2	10.3
2	1.387	5741	2.002	852	104.0	4.49	3.99
3	1.429	5660	2.001	829	101.5	1.35	1.24
4	1.440	5644	2.002	824	99.0	0.53	0.57
5	1.440	5638	2.002	824	99.1	0.48	0.49
6	1.441	5642	2.002	824	98.5	0.48	0.47
7	1.441	5639	2.001	823	99.3	0.46	0.47
8	1.441	5643	2.000	823	97.7	0.46	0.47

fission cross-sections are adjusted to the computed eigenvalue k_{eff} to achieve the initial core criticality. Region R in the layout shown in Fig. 9 initially corresponds to the same cross-sections as region 3. Then, the thermal absorption cross-section Σ_{a2} in R linearly reduces during the first two seconds until it matches the cross-section of region 4, simulating the control rod movement. Consequently, the reactor power P increases reaching a peak at time t_1 , and then decreases due to the activated temperature feedback. As the control rod continues to eject, the excessive reactivity surpasses the Doppler effect at time t_2 , where the second peak is observed. Then, the reactor power decreases while the temperature continues to increase.

Fig. 11 depicts the average reactor core power $\bar{P}$ and fuel temperature $\bar{T}$ as functions of time. The transient was computed with a relative tolerance of $\text{RTOL} = 10^{-5}$ and an initial time step of $h_0 = 10^{-6} \text{ s}$. The time step and order of the BDF method increase with each subsequent step, keeping the local truncation error of simulation within the specified relative tolerance. The calculations were performed using FEM orders from 1 to 8. The presented power and temperature were computed using the 4th order and they are compared to the reference points calculated by a finite-difference code with a fine mesh, as provided by ANL-7416/2 (Finnemann, 1977).

The increasing the order beyond 4 does not significantly improve the simulation. This is illustrated by Table 5 showing the power peaks P_1 and P_2 at times t_1 and t_2 , and the temperature peak T_3 at the end of simulation time, depending on the FEM order. In addition, the Root-Mean-Square (RMS) errors of the power and temperature assembly

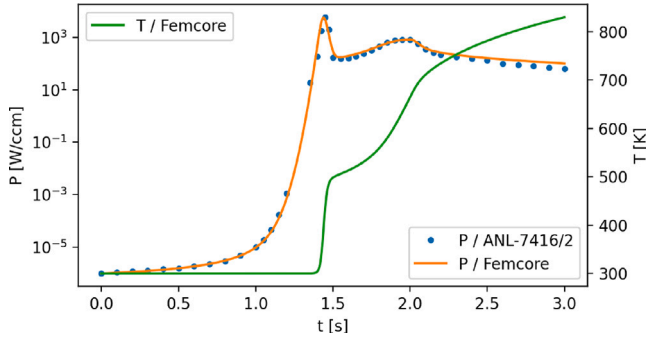


Fig. 11. Average power and temperature for LRA 2D.

Table 6
LRA 2D simulation results.

Parameter	FEMCORE	FEM	SKETCH-N	Reference
Eigenvalue (rods-in)	0.99639	0.99639	0.99635	0.99633
Eigenvalue (rods-out)	1.01550	1.01554	–	1.01546
Time to first peak (s)	1.439	1.432	1.437	1.436
Power at first peak (W/cm ³)	5641	5581	5567	5411
Power at second peak (W/cm ³)	820	–	818	784
Power at $t = 3$ s (W/cm ³)	98.7	–	103	96.2
Average fuel temp. at $t = 3$ s (K)	1122	–	1019	1087
Peak fuel temp. at $t = 3$ s (K)	3068	–	3059	2948

distributions δP_3 and δT_3 at the end of simulation are presented. As can be concluded, the 4th FEM degree produces the converged solutions within 0.6% of the reference.

Table 6 presents a comparison of FEMCORE calculations with a FEM code developed by Senecal (Senecal and Ji, 2018) and the nodal diffusion code SKETCH-N (Zimin and Ninokata, 1998). The eigenvalues for the rods-in and rods-out cases were computed at room temperature $T = 300$ K, without temperature feedback. Overall, the parameters show good agreement with the reference and the nodal code. The differences observed are due to variations in the computational mesh and approximation functions within homogeneous cells.

Fig. 12 shows distribution of neutrons for fast and thermal energy group at the end time t_3 of the simulation process. The peak at the location of withdrawn control rod is observed as compared to the initial steady-state distributions presented in Fig. 10. The detail comparison of the power and temperature by assemblies to the reference values of ANL-7416/2 benchmark (Finnemann, 1977) are presented in Appendix B. The maximum deviation of 1.6% power is observed for the central assembly, attributed to the low power at this point. The deviation within the high-power fuel assemblies near the ejected control rod is approximately 0.6%. The largest discrepancy in temperature, approximately 2.5%, is observed for the peripheral fuel assemblies.

3.3.4. Iteration performance

In this section we analyse the performance of LRA-2D in terms of the number of global, constituent and inner iterations, referred to the BDF, GS and CG methods, respectively. The number of iterations is depicted in Table 7, where FEM denotes the order of B-splines, REG and REJ denotes the number of regular and rejected time steps of BDF, NLS denotes the number of constituent iterations required to solve the in-step coupled system of neutronics and heat equations, GS/An denotes the number of iterations of the GS method accelerated by the Anderson's method, MUL denotes the number of matrix-vector products, that is the number of iterations of the ILU(0)-preconditioned CG method. The last three columns, GS/Ch and GS/At and BiCG, indicate the ratio of the matrix-by-vector multiplications of three different methods to the GS/An method. As can be seen, the same iteration trends are observed here as in the TWIGL-2D benchmark, while GS/Ch and GS/At methods perform significantly worse.

Table 7

Performance of in-step solvers for LRA-2D.

FEM	BDF Steps		Num. of iterations			MUL ratio to GS/An		
	STP	REJ	NLS	GS/An	MUL	GS/Ch	GS/At	BiCG
1	403	5	1038	11874	109953	7.6	13.5	1.3
2	485	12	1276	12707	132532	6.7	12.2	1.6
3	443	12	1164	12504	136966	7.0	12.1	1.5
4	399	7	1029	11716	140106	7.7	12.6	1.6
5	405	5	1041	11874	149251	7.3	12.5	1.6
6	396	7	1013	11501	158252	7.6	13.3	1.6
7	421	9	1086	12010	170313	7.5	12.7	1.5
8	412	7	1062	12058	181587	7.1	12.4	1.5

The iteration performance of the BDF method is illustrated by four graphics in Fig. 13. The graphics depict the history of the local truncation error e_h calculated according to Eq. (44), time step size h , integration order q , and the number of in-step constituent iterations N required for solving the non-linear problem per each time step.

The local truncation error exhibits significant variations in magnitude, yet consistently remains within the unit interval, as required by the BDF step control method. Time step increases from the initial value 0.1 ms to about 100 ms during the first stage of the transient process, when the power change is low. Then, step size drops up to 1 ms at the moments $t_1 \approx 1.5$ s and $t_2 \approx 2$ s of the two power peaks P_1 and P_2 . These moments are characterized by the high magnitudes of the local truncation error and several rejections of a time step. BDF starts from the first order and then rapidly advances to the fifth order, where it predominantly stays. At the end of the simulation, BDF step size and order reach the highest values, indicating the power and temperature transients are smooth and well interpolated by a 5th order polynomial. The number of in-step iterations is dependent on the iteration method. The Anderson method outperforms the Picard iteration, demonstrating a maximum of four iterations, while the Picard method requires between 5 to 10 iterations. Interestingly, the selection of the method practically does not influence the BDF step size and order histories.

Fig. 14 depicts (A) the number of time steps, (B) the number of constituent iterations, and (C) the average number of constituent iterations per step required to solve the LRA-2D benchmark using BDF with various relative tolerances (RTOL). Several simulations were conducted using the Picard method with maximum BDF orders of 2 (P2) and 5 (P5), and Anderson's method with maximum BDF orders of 2 (A2) and 5 (A5). All simulations employed 4th degree FEM. As seen in the graphs, the number of time steps and constituent iterations increase as the relative tolerance decreases. The number of Picard iterations significantly exceeds the number of Anderson iterations, with the difference becoming more pronounced at higher solution accuracy.

To clarify the behavior of these graphics, we compare the number of time steps n_1 and n_2 required for simulation of LRA-2D using two different relative tolerances r_1 and r_2 for the BDF method. The number of time steps is inversely proportional to the step size, $n_1 \sim 1/h_1$, since the time simulation interval is fixed. Using Eqs. (44) and (45), we deduce the following relationships:

$$\frac{h_1}{h_2} \sim \frac{n_2}{n_1} \sim \left(\frac{r_1}{r_2} \right)^{\frac{1}{1+q}}, \quad (53)$$

where the BDF method's order q is approximately 5, as follows from the order history graph in Fig. 13. For example, if $r_1 = 10^{-2}$ and $r_2 = 10^{-8}$, the difference in the number of time steps $n_2/n_1 \sim 10$, which agrees with graph (A) of Fig. 14.

3.3.5. Time performance

In this section, we discuss the performance of FEMCORE in terms of CPU time for the LRA 2D benchmark calculated using Anderson's method. Table 8 shows the percentage breakdown of CPU time across

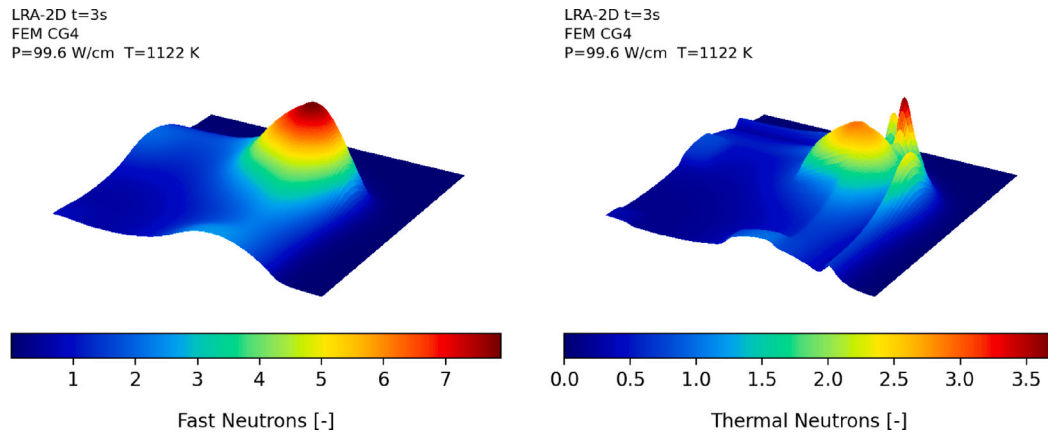


Fig. 12. LRA-2D neutron flux at 3 s.

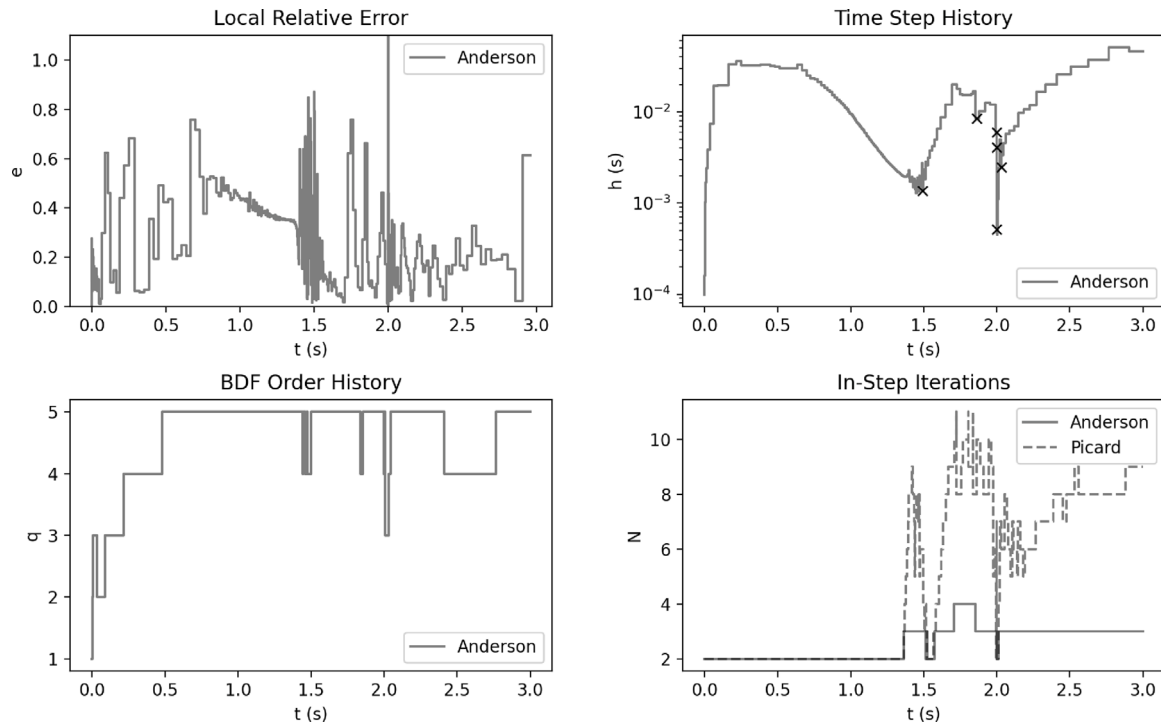


Fig. 13. Performance of BDF for LRA-2D.

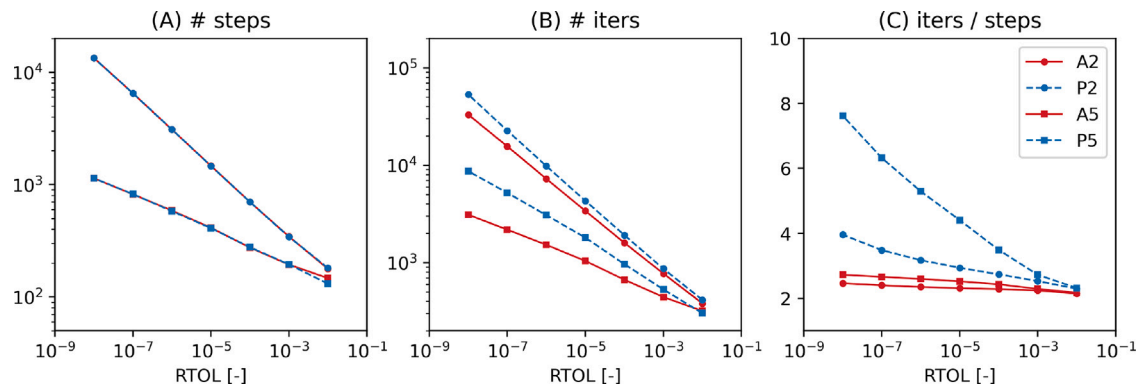


Fig. 14. Number of steps and constituent iterations.

code routines for FEM orders from 1 to 8. The routines are grouped into four major counters: matrices update (UPD), fixed point iteration

(FPI), preconditioning (PRE), and matrix–vector multiplication (MUL). In addition, the number of degrees-of-freedom (DOF), the number of

Table 8
Code performance for LRA 2D.

FEM Order	DOF [u^i]	N_1 [e^{ij}]	N_2 [e_e^{ij}]	CPU Fraction, %				CPU, s
				UPD	FPI	PRE	MUL	
1	144	1156	1936	6	9	34	52	1.4
2	529	7921	9801	10	8	40	42	4.7
3	1156	27556	30976	10	5	39	46	12.2
4	2025	70225	75625	9	4	40	47	29.1
5	3136	148996	156816	11	3	43	43	62.2
6	4484	279841	290521	11	2	40	47	121.7
7	6084	481636	495616	12	2	38	49	237.0
8	7921	776161	793881	11	1	38	50	438.1

non-zero elements N_1 and N_2 of terms e^{ij} and e_e^{ij} are presented. The total CPU times in seconds required for simulation of the LRA 2D benchmark are presented in the last column.

The UPD counter accumulates the time spent assembling the finite element matrices according to Eq. (14), which are diffusion D_g^{ij} , collision Σ_g^{ij} , scattering $S_{gg'}^{ij}$, and fission production F_g^{ij} . In the LRA 2D benchmark, only the collision matrix needs to be updated at each constituent iteration. However, to generalize this illustration, UPD accounts for updating all finite element matrices required. The arithmetic complexity of assembling one finite element matrix is $O(N_2)$. Note, this complexity and the overall CPU time do not account the computation of the integrands e_e^{ij} , δ_e^{ij} and σ^{ij} . The FPI counter accumulates the time spent computing the least-square matrix given by Eq. (43) for Anderson's method, that is the dot products of the vectors computed at the last m iteration vectors, where m is the Anderson's method depth. There is no need to compute all matrix entries at each iteration, but only the rows and columns corresponding to the updated vector. Therefore, the arithmetic complexity of this operation is $O(m \cdot DOF)$. The PRE and MUL counters stand for the preconditioning and vector-matrix multiplication called in the CG iterations. The complexity of each of these two operations is $O(N_1)$.

As the FEM order increases, DOF increases linearly, while the number of non-zero elements N_1 and N_2 increases exponentially. Consequently, the relative contribution of FPI decreases, whereas the relative fractions of UPD, PRE, and MUL remain approximately the same for all orders. The total CPU time of the simulations grows exponentially with factor from 2 to 3 in line with the number of non-zero elements.

4. Conclusions

In this study, we developed an algorithm to solve the multigroup diffusion neutron kinetics equations using high-order methods. The algorithm employs B-spline FEM for spatial discretization with orders ranging from 1 to 8 and the multi-step Nordsieck's BDF implicit method for time integration with orders from 1 to 5. The elementary FEM integrals e_e^{ij} , δ_e^{ij} , and σ^{ij} , representing collision, diffusion, and leakage processes, respectively, are assembled using the open-source library Nutils (van Zwieten et al., 2022) for arbitrary mesh and geometry. The entire system of multigroup diffusion equations is assembled and solved using the in-house code FEMCORE (Cherezov et al., 2024). The in-step system of multi-group diffusion equations is solved using the block GS method, enhanced by Anderson's method for better convergence. The neutron diffusion equation within a specific energy group is solved with the ILU(0)-preconditioned CG method.

The algorithms were verified for various orders of FEM and BDF methods using the point-kinetics benchmark SKINATH (Ganapol, 2013) and two spatial benchmarks, TWIGL-2D (Hageman and Young, 1981) and LRA-2D (Finnemann, 1977). The assessment of the performance of the implemented methods and algorithms showed that:

1. Anderson's method significantly accelerates both the constituent and group inner iterations for the in-step non-linear system

and the multi-group diffusion equation, respectively. For constituent iterations, it converges up to 8 times faster than the Picard method, depending on the BDF scheme order, as seen in Fig. 14. For inner block GS iterations applied to the multi-group equation, Anderson's method is 5 to 13 times faster than the Chebyshev and Aitken acceleration methods, as seen in Table 3 for TWIGL-2D and Table 7 to LRA-2D. In addition, the method is competitive with a BiCG solver applied to the entire multi-group matrix.

2. The 5th order BDF method with automatic order and step size selection significantly enhances the performance of the transient solver. It achieves a 10 times improvement compared to the 1st and 2nd order schemes, as shown in Fig. 7 for TWIGL-2D. Additionally, using Anderson's method for solving in-step equations improves the stability of step size and order selection compared to the Newton-Raphson method implemented in LSODE (Radhakrishnan and Hindmarsh, 1993), as illustrated in Figs. 4 and 5 for SKINATH.
3. Increasing the FEM order affects simulation complexity through: a) the convergence of the ILU(0)-preconditioned CG solver due to the larger condition numbers of FEM matrices, and b) the complexity of matrix-by-vector multiplication due to the larger number of non-zero elements in FEM matrices. However, the FEM order does not significantly affect the number of global, constituent, and inner iterations, represented by the BDF, Anderson's, and block GS methods, respectively, as shown in Table 3 for TWIGL-2D and Table 7 for LRA-2D.
4. The total CPU time for simulations is almost equally divided between preconditioning procedure and matrix-by-vector multiplication, with around 10% allocated to matrix update operations (this does not include the computation of the integrands). Consequently, the CPU time increases exponentially with the FEM order due to the rise in the number of non-zero elements in the FEM matrices, as illustrated in Table 8 for LRA-2D.

Note that these conclusions are drawn from the analysis of small two-dimensional benchmark tests, and need to be reassessed for full-core three-dimensional cores.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used ChatGPT 3.5 in order to correct English grammar and improve the readability of the text without modifying any technical statements of the original text. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Acknowledgments

This work was partly funded by the Swiss Nuclear Safety Inspectorate ENSI (CTR00943) and conducted within the framework of the STARS program (<http://www.psi.ch/stars>).

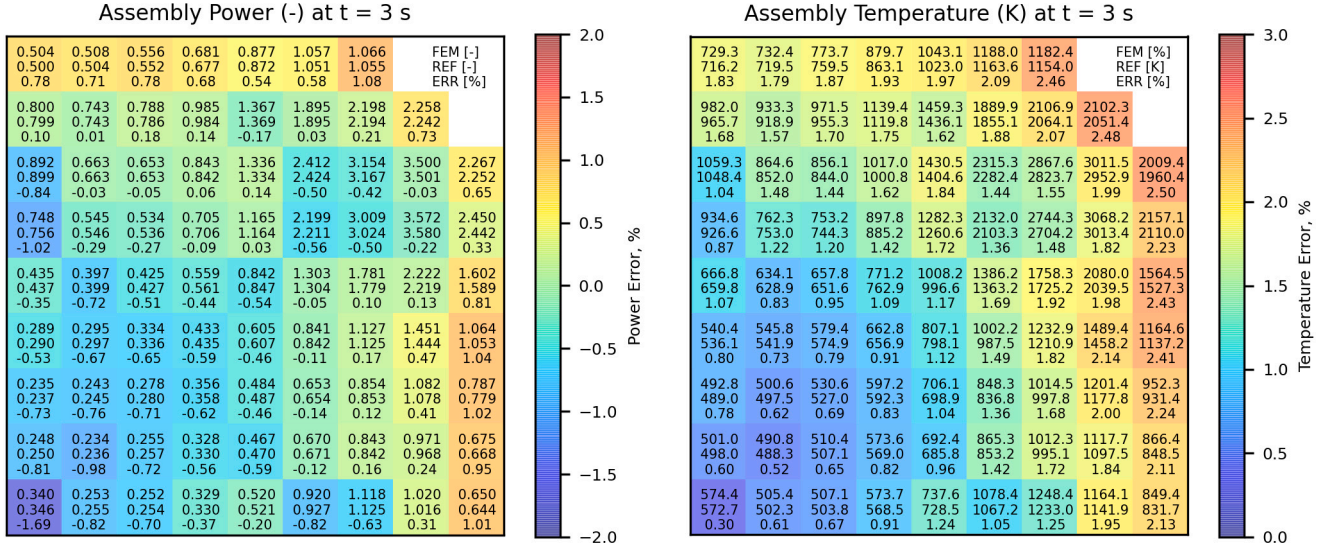


Fig. 15. Power and temperature distributions of LRA-2D calculated by FEMCORE (FEM) and compared to the reference values (REF) provided by (Finnemann, 1977), and their corresponding errors (ERR).

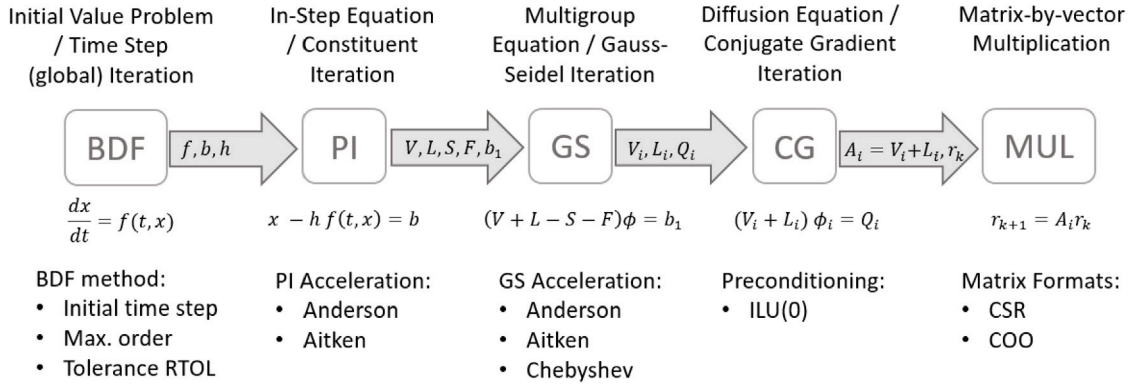


Fig. 16. FEMCORE Transient Simulation Algorithm.

Appendix A

Index notation is a convenient way to express and manipulate multi-indexed quantities, including tensors in differential geometry and multi-dimensional arrays in scientific computing (Papastavridis, 1999; Åhlander, 2002). The compactness of this notation comes from two conventions: the range convention and the Einstein summation convention. The range convention implies the free indices loop over all values in a specific range. The Einstein summation convention implies the repeated indices are summed over the range. As different authors may interpret index notation differently, we specify the rules used in this paper here.

The notation we use applies only to multi-dimensional arrays, distinguishes between upper and lower indices, and allows the same index to appear twice in the lower or upper position. The primary rules are:

1. The terms are indexed with upper and lower indices. For instance, assuming indices i and j vary from 1 to n , term x_i denotes a one-dimensional array $(x_1, x_2, \dots, x_n)$, and term A_i^j denotes a two-dimensional array of rank n .
2. The same symbol appearing twice (one upper, one lower) within a term indicates summation over that index. For instance, $x_i y^i = \sum_{i=1}^n x_i y_i$ denotes the dot product between two arrays.
3. One can freely raise and lower an index for a symmetric array. For instance, arrays A_{ij} and A_i^j are equivalent since they contain

the same entries, which can be uniquely accessed via indices i and j .

4. If an index appears twice as a lower or upper index, summation is not implied. For instance, expression $x_i y^i = a$ is understood as one equation $\sum_{i=1}^n x_i y_i = a$, while expression $x_i y_i = a$ is understood as a system of n equations, $x_1 y_1 = a, \dots, x_n y_n = a$.
5. Summation can occur multiple times within a term. For instance, $A^i B_i^k C_{kj} = \sum_i \sum_k A^i B_i^k C_{kj}$.
6. The same rules are applied to fractions. For instance, $\frac{a_i}{b^i} = \sum_{i=1}^n \frac{a_i}{b^i}$.
7. The same rules are applied to differentiation. For instance, $\frac{\partial J^i}{\partial x_i}$ denotes the divergence of vector J^i , while $\frac{\partial J^i}{\partial x^i}$ is interpreted as a single term for each $i = 1, \dots, n$, as this index appears twice as an upper index.
8. An indexed term used as a function argument is understood as a list of arguments. For instance, $f(x_i) = f(x_1, x_2, \dots, x_n)$ is the scalar function of n arguments.

Appendix B

Fig. 15 depict the assembly average power and temperature, and their deviations from the reference solution of the LRA-2D benchmark (Finnemann, 1977) at time 3 s. There are three numbers in

each cell, which are the computed value by FEMCORE (FEM), the reference value (REF), and their relative difference (ERR) given in %. The computed and reference power distributions were normalized to unite average. The computations by FEMCORE were performed using 4th order FEM and relative tolerance 10^{-5} in BDF.

Appendix C

Fig. 16 presents the FEMCORE algorithm for transient simulation. The initial value problem is integrated by the BDF method with arbitrary order and time step size, both selected automatically based on the relative tolerance error RTOL. The BDF routine invokes the Picards iterations (PI) for solution of the non-linear in-step equation that couples neutronics, heat transfer and other reactor models. PI are accelerated by Anderson's method and optionally by Aitken's method. The neutronics and heat transfer in-step equations are solved individually, with the power and temperature exchange during the PI. The neutronics component involves solving a linear multi-group diffusion equation with an external neutron source, which is solved using the GS method, accelerated by the Anderson, Aitken or Chebyshev methods. The diffusion equation for each energy groups requires the CG iterations, accelerated by the preconditioning via ILU(0) factorization. Each CG iteration calls two routines: applying the preconditioning matrix (not shown in the chain) and performing the matrix-by-vector multiplication (MUL) in either the Compressed Storage Row (CSR) format or, optionally, coordinate (COO) format.

References

- Abdulle, A., 2015. Explicit stabilized Runge-Kutta methods. In: *Encyclopedia of Applied and Computational Mathematics*. Springer, Berlin Heidelberg, pp. 460–468.
- Aboanber, A.E., Hamada, Y.M., 2009. Computation accuracy and efficiency of a power series analytic method for two- and three- space-dependent transient problems. *Prog. Nucl. Energy* 51 (3), 451–464.
- Aggarwal, B., 2006. B-spline finite elements for plane elasticity problems. (Master's thesis). Texas A&M University.
- Åhländer, K., 2002. Einstein summation for multidimensional arrays. *Comput. Math. Appl.* 44 (8), 1007–1017.
- Al Daas, H., Grigori, L., Hénon, P., Ricoux, P., 2018. Enlarged GMRES for solving linear systems with one or multiple right-hand sides. *IMA J. Numer. Anal.* 39 (4), 1924–1956.
- Anistratov, D.Y., Cornejo, L.R., Jones, J.P., 2017. Stability analysis of nonlinear two-grid method for multigroup neutron diffusion problems. *J. Comput. Phys.* 346, 278–294.
- Avvakumov, A.V., Strizhov, V.F., Vabishchevich, P.N., Vasilev, A.O., 2019. Automatic time step selection for numerical solution of neutron diffusion problems. In: Dimov, I., Faragó, I., Vulkov, L. (Eds.), *Finite Difference Methods. Theory and Applications*. Springer International Publishing, Cham, pp. 145–152.
- Avvakumov, A.V., Vabishchevich, P.N., Vasilev, A.O., Strizhov, V.F., 2016. Solution of the neutronics code dynamic benchmark by finite element method. *AIP Conf. Proc.* 1773 (1), 110003.
- Babcsányi, B., Bartók, T., Kis, D.P., 2020. Finite element solution of the time-dependent SP3 equations using an implicit integration scheme. *Kerntechnik* 85 (4), 292–300.
- Babuska, I., Strouboulis, T., 2001. The finite element method and its reliability.
- Baudron, A.-M., Lautard, J.-J., 2007. MINOS: A simplified pn solver for core calculation. *Nucl. Sci. Eng.* 155 (2), 250–263.
- Baudron, A.-M., Lautard, J.-J., Maday, Y., Mula, O., 2014. MINARET: Towards a time-dependent neutron transport parallel solver. In: *SNA + MC 2013 - Joint International Conference on Supercomputing in Nuclear Applications + Monte Carlo*.
- Boffie, J., Pounders, J., 2018. An adaptive time step control scheme for the transient diffusion equation. *Ann. Nucl. Energy* 116, 280–289.
- Bousquet, J., Seubert, A., Henry, R., 2020. New finite element neutron kinetics coupled code system FENNECS/ATHLET for safety assessment of (very) small and micro reactors. *J. Phys. Conf. Ser.* 1689 (1), 012008.
- Brenner, S.C., Carstensen, C., 2004. *Encyclopedia of Computational Mechanics, Volume 1 Fundamentals, Chapter 4 Finite Element Methods*. Wiley.
- Brenner, S.C., Scott, L.R., 1994. *The Mathematical Theory of Finite Element Methods*. Springer.
- Bru, R., Genstar, D., Marin, J., Verdú, G., Mas, J., Manteuffel, T., 2002. Iterative schemes for the neutron diffusion equation. *Comput. Math. Appl.* 44 (10), 1307–1323.
- Cacuci, D.G., 2010. *Handbook of Nuclear Engineering*. Springer.
- Cai, Y., Peng, X., Li, Q., Wang, K., Qin, X., Guo, R., 2019. The numerical solution of space-dependent neutron kinetics equations in hexagonal-z geometry using backward differentiation formula with adaptive step size. *Ann. Nucl. Energy* 128, 203–208.
- Calloo, A., Le Tellier, R., Couyras, D., 2023. Anderson acceleration and linear diffusion for accelerating the k-eigenvalue problem for the transport equation. *Ann. Nucl. Energy* 180, 109406.
- Carreño, A., Vidal-Ferrándiz, A., Ginestar, D., Verdu, G., 2021. Adaptive time-step control for modal methods to integrate the neutron diffusion equation. *Nucl. Eng. Technol.* 53 (2), 399–413.
- Cherezov, A., Park, J., Kim, H., Choe, J., Lee, D., 2020. A multi-physics adaptive time step coupling algorithm for light-water reactor core transient and accident simulation. *Energies* 13 (23).
- Cherezov, A., Vasiliev, A., Ferroukhi, H., 2023. Acceleration of nuclear reactor simulation and uncertainty quantification using low-precision arithmetic. *Appl. Sci.* 13 (2).
- Cherezov, A., Vasiliev, A., Ferroukhi, H., 2024. Low precision preconditioning for solving neutron diffusion eigenvalue problem by finite element method. *Ann. Nucl. Energy* 206, 110575.
- Ern, A., Guermond, J.L., 2004. *Theory and Practice of Finite Elements*. Applied Mathematical Sciences. Springer.
- Eyert, V., 1996. A comparative study on methods for convergence acceleration of iterative vector sequences. *J. Comput. Phys.* 124, 271–285.
- Fang, H., Saad, Y., 2009. Two classes of multisecond methods for nonlinear acceleration. *Numer. Linear Algebra Appl.* 16, 197–221.
- Ferguson, J.A., Kópházi, J., Eaton, M.D., 2022. NURBS enhanced virtual element methods for the spatial discretization of the multigroup neutron diffusion equation on curvilinear polygonal meshes. *J. Comput. Theoret. Transp.* 51 (4), 145–204.
- Finnemann, H., 1977. Argonne Code Center: Benchmark problem book. (2), <http://dx.doi.org/10.2172/12030251>, Argonne National Laboratory, United States, URL <https://www.osti.gov/biblio/12030251>.
- Fiorina, C., Clifford, I., Kelm, S., Lorenzi, S., 2022. On the development of multi-physics tools for nuclear reactor analysis based on OpenFOAM®: state of the art, lessons learned and perspectives. *Nucl. Eng. Des.* 387, 111604.
- Fiorina, C., Kerkar, N., Mikityuk, K., Rubiolo, P., Pautz, A., 2016. Development and verification of the neutron diffusion solver for the GeN-Foam multi-physics platform. *Ann. Nucl. Energy* 96, 212–222.
- Ganapol, B.D., 2013. A highly accurate algorithm for the solution of the point kinetics equations. *Ann. Nucl. Energy*.
- Ge, J., Zhang, D., Tian, W., Wang, K., Qiu, S., Su, G., 2015. Steady and transient solutions of neutronics problems based on finite volume method (FVM) with a CFD code. *Prog. Nucl. Energy* 85, 366–374.
- Gear, C.W., 1971. *Numerical Initial Value Problems in Ordinary Differential Equations*. Prentice Hall, Englewood Cliffs, NJ.
- Gehin, J.C., 1992. A quasi-static polynomial nodal method for nuclear reactor analysis.
- Ginestar, D., Verdú, G., Vidal, V., Bru, R., Marín, J., Muñoz-Cobo, J., 1998. High order backward discretization of the neutron diffusion equation. *Ann. Nucl. Energy* 25 (1), 47–64.
- González-Pintor, S., Ginestar, D., Verdú, G., 2009. High order finite element method for the lambda modes problem on hexagonal geometry. *Ann. Nucl. Energy* 36 (9), 1450–1462.
- Greiner, N., Madiot, F., Gorsse, Y., Patricot, C., Campioni, G., 2023. A new calculation strategy for molten salt reactor neutronic-thermal-hydraulic analysis implemented with APOLLO3® and TRUST/TrioCFD. *Nucl. Sci. Eng.* 197 (12), 3000–3021.
- Grigori, L., Gilbert, J.R., Cosnard, M., 2008. Symbolic and exact structure prediction for sparse gaussian elimination with partial pivoting. *SIAM J. Matrix Anal. Appl.* 30 (4), 1520–1545, Cited by: 6.
- Gupta, A., Modak, R., Gupta, H., Kumar, V., Bhatt, K., 2005. Parallelised krylov subspace method for reactor kinetics by iqs approach. *Ann. Nucl. Energy* 32 (15), 1693–1703.
- Hackemack, M.W., 2021. Discontinuous Galerkin solutions for elliptic problems on polygonal grids using arbitrary-order Bernstein-Bézier functions. *J. Comput. Phys.* 437, 110293.
- Hageman, L.A., Yasinsky, J.B., 1969. Comparison of alternating-direction time-differencing methods with other implicit methods for the solution of the neutron group-diffusion equations. *Nucl. Sci. Eng.* 38 (1), 8–32.
- Hageman, L.A., Young, D.M., 1981. *Applied Iterative Methods*. Academic Press, New York.
- Hairer, E., Norsett, S.P., Wanner, G., 1987. *Solving Ordinary Differential Equations: I. Nonstiff Problems*. Springer Series in Computational Mathematics, Berlin.
- Hairer, E., Wanner, G., 1996. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. Springer Series in Computational Mathematics.
- Hamilton, S., Berrill, M., Clarno, K., Pawłowski, R., Toth, A., Kelley, C., Evans, T., Philip, B., 2016. An assessment of coupling algorithms for nuclear reactor core physics simulations. *J. Comput. Phys.* 311, 241–257.
- Hébert, A., 2008. A Raviart-Thomas-Schneider solution of the diffusion equation in hexagonal geometry. *Ann. Nucl. Energy* 35 (3), 363–376.
- Hindmarsh, A., 1983. ODEPACK, a Systematized Collection of ODE Solvers, in *Scientific Computing*. Elsevier, ISBN13: 978-0444866073, LC: Q172.S35.
- Holling, K., 2003. *Finite Element Methods with B-Splines*. Society for Industrial and Applied Mathematics, p. 125.

- Jaradat, M.K., Choi, N., Abou-Jaoude, A., 2024. Verification of Griffin-Pronghorn-coupled multiphysics code system against CNRS molten salt reactor benchmark. *Nucl. Sci. Eng.* 1–34.
- Jaradat, M.K., Sik Yang, W., 2023. An adaptive time-stepping control algorithm for molten salt reactor transient analyses. *Ann. Nucl. Energy* 190, 109880.
- Kepisty, G., Patricot, C., Broc, D., Campioni, G., 2017. SFR mechanical scenarios and neutron transport transients with CAST3M code. *Ann. Nucl. Energy* 101, 226–236.
- Kotchoubey, J., 2015. POLCA-T neutron kinetics model benchmarking. (Master of Science Thesis). KTH Royal Institute of Technology.
- Kulikov, G., 2009. On quasi-consistent integration by Nordsieck methods. *J. Comput. Appl. Math.* 225 (1), 268–287.
- Lamarsh, J.R., 1966. *Introduction to Nuclear Reactor Theory*. Addison-Wesley.
- Langenbuch, S., Velkov, K., Pevec, D., Grgic, D., 1996. Capability of the QUABOX/CUBBOX-ATHLET Coupled Code System. International conference: Nuclear option in countries with small and medium electricity grid, Croatia.
- Lee, C., Jung, Y.S., Park, H., Shemon, E.R., Ortensi, J., Wang, Y., Laboure, V.M., Prince, Z.M., 2021. Griffin software development plan.
- Lewis, E.E., 2008. *Fundamentals of Nuclear Reactor Physics*. Elsevier Inc..
- Io Muzio, S., Seubert, A., 2024. Implementation of the steady state simplified P3 (SP3) transport solver in the finite element neutronic code FENNECS, part 1: Theory. *Ann. Nucl. Energy* 110303.
- Ma, Y., Wang, Y., Yang, J., 2021. ntkFoam: An OpenFOAM based neutron transport kinetics solver for nuclear reactor simulation. *Comput. Math. Appl.* 81, 512–531. Development and Application of Open-source Software for Problems with Numerical PDEs.
- Nordsieck, A., 1962. On numerical integration of ordinary differential equations. *Math. Comp.* 16 (22).
- Papastavridis, J., 1999. *Tensor Calculus and Analytical Dynamics*, first ed. Routledge, p. 646.
- Phillips, T.R., Heaney, C.E., Chen, B., Buchan, A.G., Pain, C.C., 2023. Solving the discretised neutron diffusion equations using neural networks. *Internat. J. Numer. Methods Engrg.* 124 (21), 4659–4686.
- Piegl, L., Tiller, W., 1997. *The NURBS Book*. Springer Berlin, Heidelberg, p. 646.
- Radhakrishnan, K., Hindmarsh, A., 1993. Description and use of LSODE, the Livermore solver for ordinary differential equations. Tech. Rep. UCRL-ID-113855, Lawrence Livermore National Laboratory.
- Ragusa, J.C., 2015. Discontinuous finite element solution of the radiation diffusion equation on arbitrary polygonal meshes and locally adapted quadrilateral grids. *J. Comput. Phys.* 280, 195–213.
- Rintala, A., Lauranto, U., 2023. Time-dependent neutronics model of nodal neutronics program ANTS. *Ann. Nucl. Energy* 190, 109868.
- Saad, Y., 2003. Iterative methods for sparse linear systems: Second edition. In: *Other Titles in Applied Mathematics*, Society for Industrial and Applied Mathematics (SIAM, 3600 Market Street, Floor 6, Philadelphia, PA 19104).
- Schmid, W., Wagner, F., 1995. Numerical solution of the neutron diffusion equation—Adaptive concepts in time and space. In: Hackbusch, W., Wittum, G. (Eds.), *Numerical Treatment of Coupled Systems: Proceedings of the Eleventh GAMM-Seminar*, Kiel, January 20–22, 1995. Vieweg+Teubner Verlag, Wiesbaden, pp. 163–174.
- Scott, J., Tuma, M., 2023. Sparse LU factorizations. *Necas Center Series* 89–111.
- Senecal, J.P., Ji, W., 2018. Development of an efficient tightly coupled method for multiphysics reactor transient analysis. *Prog. Nucl. Energy* 103, 33–44.
- Shen, Q., Kochunas, B., 2024. Practical considerations for the adoption of Anderson acceleration in nonlinear diffusion accelerated transport. *Ann. Nucl. Energy* 199, 110330.
- Shim, C.B., Jung, Y.S., Joo, H.G., Yoon, J.I., 2011. Application of backward differentiation formula to spatial reactor kinetics calculation with adaptive time step control. *Nucl. Eng. Technol.* 43 (6), 531–546.
- Smith, K., 1979. Msc. Thesis: An analytic nodal method for solving the two-group, multidimensional, static and transient neutron diffusion equations. tech. rep., Massachusetts Institute of Technology.
- Stacey, W.M., 2007. *Nuclear Reactor Physics*. John Wiley and Sons.
- Sutton, T., Aviles, B., 1996. Diffusion theory methods for spatial kinetics calculations. *Prog. Nucl. Energy* 30 (2), 119–182.
- Tibera, M., Lathouwers, D., Kloosterman, J.L., 2020. A multi-physics solver for liquid-fueled fast systems based on the discontinuous galerkin fem discretization. *Prog. Nucl. Energy* 127, 103427.
- Vidal-Ferrándiz, A., Carreño, A., Ginestar, D., Demazière, C., Verdú, G., 2020a. Neutronic simulation of fuel assembly vibrations in a nuclear reactor. *Nucl. Sci. Eng.* 194 (11), 1067–1078.
- Vidal-Ferrándiz, A., Carreño, A., Ginestar, D., Demazière, C., Verdú, G., 2020b. A time and frequency domain analysis of the effect of vibrating fuel assemblies on the neutron noise. *Ann. Nucl. Energy* 137, 107076.
- Vidal-Ferrándiz, A., Ginestar, D., no, A.C., Verdú, G., Dokhane, A., Verma, V., Perin, Y., Herb, J., Mylonakis, A., Demazière, C., Vinai, P., 2022. Modelling and simulations of reactor neutron noise induced by mechanical vibrations. *Prog. Nucl. Energy* 177, 109300.
- Vidal-Ferrándiz, A., González-Pintor, S., Ginestar, D., Verdú, G., Demazière, C., 2017. Schwarz type preconditioners for the neutron diffusion equation. *J. Comput. Appl. Math.* 309, 563–574.
- Wang, Y., Ragusa, J., 2009. Application of hp adaptivity to the multigroup diffusion equations. *Nucl. Sci. Eng.* 161 (1), 22–48.
- Wang, Y., Schunert, S., Ortensi, J., Laboure, V., DeHart, M., Prince, Z., Kong, F., Harter, J., Balestra, P., Gleicher, F., 2021. Rattlesnake: A MOOSE-based multiphysics multischeme radiation transport application. *Nucl. Technol.* 207 (7), 1047–1072.
- Yang, D., Chen, G., Chou, H., 1993. Application of preconditioned conjugate gradient-like methods to reactor kinetics. *Ann. Nucl. Energy* 20 (1), 9–33.
- Zhang, X., Zeng, Q., Chen, H., 2020. Development and validation of a coupled neutron diffusion-thermal hydraulic calculation procedure for fast reactor applications. *Ann. Nucl. Energy* 139, 107243.
- Zienkiewicz, O., Taylor, R., Zhu, J., 2013. *The Finite Element Method*. Elsevier.
- Zimin, V.G., Cherezov, A., 2017. Application of backward differentiation formulas to neutron kinetics problems. *Phys. At. Nucl.* 80, 1377–1386.
- Zimin, V.G., Ninokata, H., 1996. Acceleration of the outer iterations of the space-dependent neutron kinetics equations solution. *Ann. Nucl. Energy* 23 (17), 1407–1424.
- Zimin, V.G., Ninokata, H., 1998. Nodal neutron kinetics model based on nonlinear iteration procedure for LWR analysis. *Ann. Nucl. Energy* 25 (8), 507–528.
- van Zwieten, J., van Zwieten, G., Hoitinga, W., 2022. *Nutils 8.0*. Zenodo.